

ArgoAWXk8sDVPdemo

Описание проекта, архитектуры и демонстрационных сценариев

Сформировано из Markdown-документации репозитория.

Актуальный стенд: [practicum-tks на d8case.ru](https://practicum-tks.d8case.ru).

Оглавление

Оглавление	2
Как читать этот документ	14
Состав источников	14
README проекта	15
Что демонстрирует стенд	15
Компоненты	15
Архитектура	15
Структура репозитория	16
Правило ведения контекста	16
Требования	17
Быстрый локальный запуск	17
Запуск в DKP/DVP	17
Установка платформенных компонентов в проект `practicum-tks`	18
Актуальный контур `practicum-tks`	18
Расширенный DVP-контур	19
Сценарии	20
Self-service UI	20
Self-service portal в кластере	21
Что делает bootstrap	22
Проверка pod-only сценария	22
Troubleshooting	22
`run-demo-job.sh` возвращает 401	22
`demo-platform` OutOfSync	22
DVP disk долго в `Provisioning`	23
AWX долго стартует	23
Rollback и очистка	23
Паспорт стенда d8case.ru	24
Идентификаторы	24
URL	24
Платформенные объекты	24
Golden images	24
Tenant environment	25
Пользователи	25
Source of truth	25
Историческая документация	25
Текущий статус	27
Правило работы	27

Репозиторий	27
Новый стенд	27
Пользователи и RBAC	27
Golden images	28
Self-service	28
Проверки	31
Сценарии	32
PDF-документация	32
Известные особенности	33
Актуальные сценарии d8case.ru: индекс	34
Актуальные адреса	34
Порядок показа	34
Правила безопасности	34
Соответствие старым сценариям	35
00. Подготовка демонстратора	36
Цель	36
1. Репозиторий	36
2. Kubernetes context	36
3. Состояние платформы	36
4. Состояние DVP	37
5. Проверка URL	37
6. Активные заявки	37
Что сказать аудитории	37
01. Архитектура GitOps + DVP + AWX	38
Цель	38
Цепочка	38
1. Показать Git	38
2. Показать настройку Argo CD	38
3. Показать controller	39
4. Показать AWX	39
5. Показать DVP	39
Ожидаемый вывод рассказа	40
02. Управление golden images	41
Цель	41
Исходное состояние	41
1. Источник образа	41
Альтернативные source images для демонстрации	41
2. Builder v1 и v2	42
3. AWX pipeline	43

4. Опубликованные версии	43
5. Active image	43
6. Как показать выпуск v3	43
Rollback	44
Нельзя делать	44
03. Self-service через Git и YAML	45
Цель	45
1. Подготовить переменные	45
2. Создать EnvironmentRequest	45
3. Commit и push	46
4. Наблюдать Git	46
5. Наблюдать Argo CD	46
6. Наблюдать Kubernetes/DVP	46
7. Наблюдать AWX	46
8. Прочитать итоговый status	46
9. Доступ	47
Cleanup	47
04. Self-service через Web	48
Цель	48
Пользователи и профили	48
1. Открыть портал	48
2. Объяснить поля	48
Профиль стенда	48
Назначение	48
TTL	48
3. Создать стенд	49
4. Читать прогресс	49
5. Проверить результат	49
6. Показать Git	49
7. Мои стенды	50
Ожидаемый результат	50
05. Административный lifecycle Victor	51
Цель	51
Исходное состояние	51
1. Открыть портал	51
2. Показать представления	51
3. Выбрать стенд	51
4. Stop/Start/Restart	51
5. Удалить только VM	52

6. Удалить стенд целиком	52
7. Проверить аудит	52
Что нельзя делать Victor	52
06. Drift correction DVP VM	54
Цель	54
Важно	54
1. Выбрать VM	54
2. Зафиксировать desired параметры	54
3. Открыть наблюдение	54
4. Создать drift	54
5. Показать self-heal	55
6. Controlled restart	55
Граница ответственности AWX	55
Опционально: ручное удаление VM	55
07. Cleanup и troubleshooting	56
Штатный cleanup	56
Пользователь	56
Victor	56
Проверка cleanup	56
Не удалять	56
Диагностика Argo CD	56
Диагностика controller	57
Диагностика AWX	57
Диагностика порталов	57
Финальная проверка стенда	57
Исторические сценарии: 01 Initial Deploy	58
Цель	58
Исходное состояние	58
Что меняем в Git	58
Пошаговое выполнение	58
Что показывать в ArgoCD	58
Что показывать в AWX	58
Проверка через kubectl	59
Ожидаемый результат	59
Rollback	59
Пояснение для демонстратора	59
Исторические сценарии: 02 Scale Application	60
Цель	60
Исходное состояние	60

Что меняем в Git	60
Пошаговое выполнение	60
Что показывать в ArgoCD	60
Что показывать в AWX	60
Проверка через kubectl	61
Ожидаемый результат	61
Rollback	61
Пояснение для демонстратора	61
Исторические сценарии: 03 Drift Correction	62
Цель	62
Исходное состояние	62
Что меняем в Git	62
Пошаговое выполнение	62
Что показывать в ArgoCD	62
Что показывать в AWX	62
Проверка через kubectl	62
Ожидаемый результат	62
Rollback	63
Пояснение для демонстратора	63
Исторические сценарии: 04 VM Resize	64
Цель	64
Исходное состояние	64
Что меняем в Git	64
Пошаговое выполнение	64
Что показывать в ArgoCD	64
Что показывать в AWX	65
Проверка через kubectl	65
Ожидаемый результат	65
Rollback	65
Пояснение для демонстратора	65
Исторические сценарии: 05 AWX Post-Config	66
Цель	66
Исходное состояние	66
Что меняем в Git	66
Пошаговое выполнение	66
Что показывать в ArgoCD	66
Что показывать в AWX	66
Проверка через kubectl	66
Ожидаемый результат	67

Rollback	67
Пояснение для демонстратора	67
Исторические сценарии: 06 Broken Release And Rollback	68
Цель	68
Исходное состояние	68
Что меняем в Git	68
Пошаговое выполнение	68
Что показывать в ArgoCD	68
Что показывать в AWX	68
Проверка через kubectl	69
Ожидаемый результат	69
Rollback	69
Пояснение для демонстратора	69
Исторические сценарии: 07 Self-Service Tenant	70
Цель	70
Исходное состояние	70
Что меняем в Git	70
Пошаговое выполнение	70
Что показывать в ArgoCD	70
Что показывать в AWX	71
Проверка через kubectl	71
Ожидаемый результат	71
Rollback	71
Пояснение для демонстратора	71
Исторические сценарии: 08 Golden Image Management	72
Цель	72
Исходное состояние	72
Что меняем в Git	72
Пошаговое выполнение	72
1. Показать source image в Git	72
2. Применить импорт через GitOps	73
3. Проверить builder disk и builder VM	73
4. Запустить builder VM для customization	73
5. Запустить AWX customization	73
6. Выполнить validation	74
7. Опубликовать новую версию golden image	74
8. Переключить потребителей	74
Что показывать в Argo CD	75
Что показывать в AWX	75
Проверка через kubectl	75

Ожидаемый результат	75
Rollback	75
Пояснение для демонстратора	75
План рассказа для живого демо	75
0. Вводная	76
1. Показать Git как точку выбора исходного образа	76
2. Показать Argo CD как механизм импорта в DVP	76
3. Объяснить builder VM	77
4. Показать AWX как слой customization	77
5. Показать, что именно меняет AWX	77
6. Показать validation	78
7. Публикация golden image	78
8. Переключение потребителей	78
9. Финальная формулировка	78
Исторические сценарии: 09 Self-Service Environment Request	79
Цель	79
Архитектурная идея	79
Исходное состояние	79
Что меняем в Git	79
Профили	80
Web UI	80
Пошаговое выполнение	80
1. Показать каталог профилей	80
2. Сгенерировать request через UI	80
3. Создать Git request	81
4. Показать generated manifests	81
5. Проверить Argo CD	81
6. Показать access info	81
7. AWX post-config	82
Альтернативный путь: GitOps/YAML без web UI	82
1. Получить репозиторий	82
2. Показать approved catalog	82
3. Создать request YAML	82
4. Проверить diff и отправить в Git	83
5. Generated manifests	83
6. Проверить Argo CD и ресурсы	83
7. Rollback	84
Что показывать в Argo CD	84
Что показывать в AWX	84
Проверка через kubectl	84
Ожидаемый результат	84

Rollback / Cleanup	84
Пояснение для демонстратора	85
Исторические сценарии: 10 Self-Service Portal	86
Цель	86
Исходное состояние	86
Что делает пользователь	86
Смысл	86
Что делает система	86
Что показывать в Argo CD	87
Что показывать в AWX	87
Проверка через kubectl	87
Ожидаемый результат	87
Rollback	87
Пояснение для демонстратора	87
Исторические сценарии: 11 Practicum End-to-End	89
Цель	89
Исходное состояние	89
Golden image	89
Git self-service	89
Web self-service	89
TTL cleanup	90
Пояснение для демонстратора	90
Исторические сценарии: 12 DVP VM Drift Correction	91
Цель	91
Исходное состояние	91
Важное различие: operation и drift	91
Что меняем в Git	92
Пошаговое выполнение	92
1. Открыть интерфейсы	92
2. Запустить наблюдение	92
3. Создать безопасный drift	92
4. Показать временное расхождение	92
5. Дождаться восстановления	93
6. Проверить необходимость перезапуска	93
Что показывать в Argo CD	93
Что показывать в DVP	93
Что показывать в AWX	94
Проверка через kubectl	94
Опционально: удаление только VM	94
Что нельзя делать в демонстрации	95

Ожидаемый результат	95
Rollback	95
Рекомендации по RBAC	95
Пояснение для демонстратора	96
Исторические сценарии: 13 Manual Environment Lifecycle	97
Цель	97
Исходное состояние	97
Что меняем в Git	97
Пошаговое выполнение: пользователь	97
Пошаговое выполнение: Victor	97
Что показывать в Argo CD	98
Что показывать в AWX	98
Проверка через kubectl	98
Ожидаемый результат	98
Rollback	98
Пояснение для демонстратора	98
Операционный runbook	100
Контуры стенда	100
Запуск локального стенда	100
Запуск DKP/DVP профиля	100
Повторно открыть локальные интерфейсы	101
Запуск AWX job	101
Проверка Argo CD	101
Проверка pod-only сценария	101
Проверка DVP-сценария	101
Проверка tenant	102
Частые проблемы	102
401 при запуске AWX job	102
DVP VM зависла в Pending	102
AWX долго стартует	102
Argo CD Application не Synced	102
Master node scheduling	103
Rollback	103
Очистка	103
Use cases	104
1. GitOps-доставка Kubernetes-ресурсов	104
2. GitOps-доставка DVP VM	104
3. Настройка ОС после создания ресурса	104
4. Configuration drift и self-healing	105
5. Rollback через Git	105

6. Self-service tenant onboarding	105
7. Единый audit trail	105
8. Управление golden image	106
9. Self-service создание временного стенда	106
Демонстрационный сценарий	106
Подготовка	106
Шаг 1. Показать Git как source of truth	107
Шаг 2. Показать Argo CD	107
Шаг 3. Показать DVP VM	107
Шаг 4. Показать AWX	107
Шаг 5. Показать drift correction	108
Шаг 6. Показать rollback	108
Шаг 7. Показать tenant onboarding	108
Финальная формулировка	108
Self-service GitOps	109
Принцип	109
Почему это безопаснее прямого UI в кластер	109
Web UI	109
GitOps/YAML путь	109
Production evolution	110
Self-service portal	111
Адрес	111
Архитектура	111
Пользователи и группы	111
Как работает заявка	112
Что показывает UI	112
Важное ограничение демо	113
Проверка	113
Ограничение ёмкости	114
Управление жизненным циклом	114
Demo talk track	116
0. Вступление	116
1. Git: единый источник истины	116
2. Argo CD: доставка ресурсов	116
3. DVP VM: инфраструктура из Git	117
4. AWX: настройка ОС	117
5. Drift correction	117
6. Rollback	118
7. Tenant onboarding	118
8. Финал	118

Prerequisites	120
Целевая модель	120
Рабочая станция оператора	120
Kubernetes и DKP	120
DVP	121
Storage	121
Сетевой доступ и registry	122
DNS и имена	122
Git и репозитории	123
Секреты и пользователи	123
AWX	124
Проверка готовности стенда	124
Migration plan	125
0. Зафиксировать исходное состояние	125
1. Выбрать параметры нового стенда	125
2. Подготовить целевой DKP/DVP кластер	126
3. Подготовить DNS или `/etc/hosts`	126
4. Склонировать проект и создать migration branch	126
5. Заменить параметры стенда	127
6. Решить, переносить ли текущие generated self-service стенды	127
Вариант А. Перенести как есть	127
Вариант В. Очистить перед переносом	128
7. Проверить рендер manifests до установки	128
8. Подготовить и отправить Git	128
9. Запустить базовый bootstrap на новом стенде	128
10. Применить расширенный DVP Application	129
11. Настроить DNS и проверить UI	129
12. Подготовить self-service portal	130
13. Настроить AWX для DVP VM	130
14. Настроить golden image сценарий	131
15. Проверить демонстрационные сценарии	131
16. Критерии приемки переноса	132
17. Rollback и очистка	132
18. Типовые проблемы при переносе	132
Argo CD не может собрать manifests	132
DVP запрещает менять `VirtualDisk.dataSource`	133
Portal пишет в Git, но Argo CD ничего не создает	133
Dex login возвращает 403	133
AWX не подключается к VM	133
Next steps	135

Приоритет 1. Подготовить стенд к повторному показу	135
Приоритет 2. Проверить роли и профили	135
Приоритет 3. Усилить Git workflow	136
Приоритет 4. Production hardening	136
Приоритет 5. Golden image lifecycle	136
Порядок безопасного продолжения	136
Полезные команды	137

Как читать этот документ

Документ собран из Git-репозитория `ArgoAWXk8sDVPdemo` и предназначен для демонстраторов, архитекторов и инженеров сопровождения.

Основной актуальный набор сценариев находится в разделе `d8case.ru`. Исторические сценарии сохранены отдельно: они полезны для контекста, но команды с `demo-prod` и `d8.kir.lab` не следует выполнять на `practicum`-стенде.

Используйте закладки PDF или оглавление для перехода между разделами. Каждый крупный раздел содержит исходный путь файла в репозитории.

Состав источников

Раздел	Файл
README проекта	README.ru.md
Паспорт стенда <code>d8case.ru</code>	docs/d8case/README.md
Текущий статус	docs/STATUS.md
Актуальные сценарии <code>d8case.ru</code> : индекс	scenarios/d8case/README.md
00. Подготовка демонстратора	scenarios/d8case/00-preflight.md
01. Архитектура GitOps + DVP + AWX	scenarios/d8case/01-architecture-tour.md
02. Управление golden images	scenarios/d8case/02-golden-image-lifecycle.md
03. Self-service через Git и YAML	scenarios/d8case/03-git-environment-request.md
04. Self-service через Web	scenarios/d8case/04-web-self-service.md
05. Административный lifecycle Victor	scenarios/d8case/05-victor-lifecycle.md
06. Drift correction DVP VM	scenarios/d8case/06-vm-drift-correction.md
07. Cleanup и troubleshooting	scenarios/d8case/07-cleanup-and-troubleshooting.md
Исторические сценарии: 01 Initial Deploy	scenarios/01-initial-deploy.md
Исторические сценарии: 02 Scale Application	scenarios/02-scale-application.md
Исторические сценарии: 03 Drift Correction	scenarios/03-drift-correction.md
Исторические сценарии: 04 VM Resize	scenarios/04-vm-resize.md
Исторические сценарии: 05 AWX Post-Config	scenarios/05-awx-post-config.md
Исторические сценарии: 06 Broken Release And Rollback	scenarios/06-broken-release-and-rollback.md
Исторические сценарии: 07 Self-Service Tenant	scenarios/07-self-service-tenant.md
Исторические сценарии: 08 Golden Image Management	scenarios/08-golden-image-management.md
Исторические сценарии: 09 Self-Service Environment Request	scenarios/09-self-service-environment-request.md
Исторические сценарии: 10 Self-Service Portal	scenarios/10-self-service-portal.md
Исторические сценарии: 11 Practicum End-to-End	scenarios/11-practicum-end-to-end.md
Исторические сценарии: 12 DVP VM Drift Correction	scenarios/12-dvp-vm-drift-correction.md
Исторические сценарии: 13 Manual Environment Lifecycle	scenarios/13-manual-environment-lifecycle.md
Операционный runbook	docs/operations.ru.md
Use cases	docs/use-cases.ru.md
Self-service GitOps	docs/self-service.ru.md
Self-service portal	docs/self-service-portal.ru.md
Demo talk track	docs/demo-talk-track.ru.md
Prerequisites	docs/prerequisites.ru.md
Migration plan	docs/migration-plan.ru.md
Next steps	docs/NEXT_STEPS.md

README проекта

README.ru.md

Проект показывает, как GitOps и Ansible/AWX дополняют друг друга в Kubernetes-native платформе и в Deckhouse Virtualization Platform.

Ключевая граница:

- Argo CD управляет Kubernetes/DVP-ресурсами из Git.
- AWX/Ansible выполняет настройку внутри ОС уже созданных workload'ов или виртуальных машин.

Актуальные инструкции для стенда d8case.ru:

scenarios/d8case/README.md (scenarios/d8case/README.md).

Параметры и имена объектов собраны в

docs/d8case/README.md (docs/d8case/README.md).

Сценарии в корне scenarios/, содержащие demo-prod или d8.kir.lab,

относятся к предыдущему стенду и сохранены только как исторические варианты.

В локальном Docker Desktop Kubernetes для простого запуска используются два Linux pod'а с SSH. В DKP/DVP-кластере дополнительно разворачивается настоящий минимальный DVP-контур: приложение, tenant и тестовая VM postgres-vm.

Что демонстрирует стенд

- Git является source of truth для манифестов, сценариев и Ansible-кода.
- Argo CD применяет desired state без ручного `kubectl apply`.
- DVP VM создаётся декларативно через CRD `virtualization.deckhouse.io`.
- AWX запускает Ansible job и показывает, зачем нужен procedural слой после GitOps.
- Drift, rollback, scale и tenant onboarding показываются как изменения в Git.

Компоненты

Компонент	Namespace	Роль
Gitea	gitea	Git-сервер стенда, источник манифестов и playbook'ов.
Argo CD	argocd	Синхронизирует desired state из Git.
AWX	awx	Запускает Ansible job'ы для настройки ОС/userspace.
Demo OS pods	demo-os	Локальный pod-only сценарий для Docker Desktop.
Demo platform	demo-prod	Расширенный DKP/DVP сценарий: приложение, RBAC, Ingress, заготовка monitoring-объектов, DVP VM.
Tenant example	customer-a	Пример self-service onboarding tenant'a.

Архитектура

Диаграмма Mermaid преобразована в читаемую цепочку:

1	Git commit
2	Gitea codex/demo

3	Argo CD ansible-os-pods / demo-platform
4	Kubernetes API
5	demo-os pod'ы локальный сценарий
6	demo-app namespace demo-prod
7	DVP VM postgres-vm
8	AWX post-configuration
9	Validation / marker / stdout

Git commit → Gitea codex/demo → Argo CD ansible-os-pods / demo-platform → Kubernetes API → demo-os pod'ы локальный сценарий → demo-app namespace demo-prod → DVP VM postgres-vm → AWX post-configuration → Validation / marker / stdout

Структура репозитория

Путь	Назначение
gitops/demo-manifests/	Локальный pod-only desired state для Application ansible-os-pods.
gitops/environments/prod/	Расширенный prod-контур для Application demo-platform.
gitops/environments/prod/values.yaml	Централизованные демо-параметры окружения.
gitops/environments/prod/dvp-postgres-vm.yaml	Реальный минимальный DVP manifest для postgres-vm.
gitops/environments/prod/golden-images/	Сценарий импорта исходного образа из URL, builder VM и публикации golden image.
gitops/environments/prod/tenants/customer-a/	Пример self-service tenant.
gitops/self-service/	Catalog, request и generated manifests для controlled self-service стендов.
gitops/self-service/portal/	Self-service portal с DexAuthenticator, backend и GitOps submit flow.
self-service-ui/	Одностраничный web UI для генерации GitOps self-service request.
gitops/awx/	AWX playbooks, PostSync hook и пример Secret без реальных токенов.
gitops/infrastructure/dvp/	Reference template для DVP VM.
scenarios/d8case/	Канонические сценарии действующего стенда practicum-tks на d8case.ru.
scenarios/	Исторические и универсальные варианты предыдущих стендов.
awx/os-demo-playbook.yaml	Playbook для pod-only сценария demo-os.
manifests/argocd/	Argo CD Application manifests.
manifests/dkp/	Ingress'ы для DKP-кластера.
scripts/	Bootstrap, deploy, port-forward, run-demo-job, cleanup.
docs/	Русские use cases, runbook, talk track, пререквизиты и план переноса.
docs/d8case/	Актуальный паспорт Project/namespace, URL и объектов practicum-стенда.

Правило ведения контекста

Для этого репозитория архитектурные решения фиксируются в README.md и README.ru.md. Оперативный статус хранится в docs/STATUS.md, а ближайшие планы и открытые решения - в docs/NEXT_STEPS.md.

Перед началом новой работы в репозитории сначала прочитайте:

```
docs/STATUS.md
docs/NEXT_STEPS.md
```

После значимых изменений обновляйте docs/STATUS.md. Если меняется архитектурная договорённость, обновляйте обе версии README.

Текущая стабильная точка и приоритетный backlog:

- текущее состояние (docs/STATUS.md);
- следующие шаги (docs/NEXT_STEPS.md).

На checkpoint от 9 июня 2026 года временные self-service окружения очищены по TTL, Argo CD находится в Synced/Healthy, golden image v2 активен, обе builder VM остановлены. Это безопасная точка для следующего этапа.

Требования

- kubectl
- git
- curl
- jq
- Kubernetes-кластер с default StorageClass
- Для DVP-сценария: DKP/DVP с CRD `virtualization.deckhouse.io`

Для локального сценария достаточно Docker Desktop Kubernetes. Для DKP/DVP сценария нужны IngressClass, StorageClass, DVP, Dex/OIDC и доступ к внешним образам или их внутренние зеркала.

Подробный список требований к новому стенду: docs/prerequisites.ru.md (docs/prerequisites.ru.md).

Детальный план переноса на другой DKP/DVP стенд: docs/migration-plan.ru.md (docs/migration-plan.ru.md).

Быстрый локальный запуск

```
git clone https://github.com/kirka1206/ArgoAWXk8sDVPdemo.git
cd ArgoAWXk8sDVPdemo
./scripts/bootstrap.sh
```

Скрипт поднимает Gitea, Argo CD, AWX, Application ansible-os-pods и два Linux pod'a.

Запуск AWX job:

```
./scripts/run-demo-job.sh
```

Запуск в DKP/DVP

```
./scripts/deploy-dkp.sh
```

По умолчанию скрипт настроен на текущий демо-стенд `d8.kir.lab`. Для другого стенда передайте параметры окружения:

```
CONTEXT=<target-context> \
GITEA_HOST=gitea-awx.<target-domain> \
ARGOCD_HOST=argocd-awx.<target-domain> \
AWX_HOST=awx-demo.<target-domain> \
./scripts/deploy-dkp.sh
```

На текущем стенде используются Ingress'ы:

- Gitea: <http://gitea-awx.d8.kir.lab>
- Argo CD: <http://argocd-awx.d8.kir.lab>
- AWX: <http://awx-demo.d8.kir.lab>

Для запуска AWX job в DKP используйте Ingress, а не случайный старый port-forward:

```
AWX_URL=http://awx-demo.d8.kir.lab ./scripts/run-demo-job.sh
```

Установка платформенных компонентов в проект `practicum-tks`

Для нового DKP/DVP-стенда подготовлен отдельный namespace-scoped комплект:

```
EXPECTED_CONTEXT=practicum-tks-api.d8case.ru \  
NAMESPACE=practicum-tks \  
./scripts/install-practicum-platform.sh
```

Скрипт устанавливает в namespace `practicum-tks`:

- Gitea 1.24.6, аккаунт `practicum`, репозиторий `practicum/practicum-demo`;
- Argo CD 3.4.2, локальный аккаунт `practicum-admin`;
- AWX Operator 2.19.1 и AWX 24.6.1, аккаунт `practicum-admin`;
- Ingress-объекты и минимальные default resources для служебных pod сторонних операторов.

Архитектурное решение: рабочие компоненты и их RBAC размещаются в проекте `practicum-tks`. Cluster-scoped CRD Argo CD и AWX создаются отдельно, потому что Kubernetes не поддерживает namespace-scoped CRD. Argo CD получает Role только на управление ресурсами namespace `practicum-tks`, а не всего кластера.

Адреса:

- Gitea: <http://gitea-practicum.d8case.ru>
- Argo CD: <http://argocd-practicum.d8case.ru>
- AWX: <http://awx-practicum.d8case.ru>
- Self-service: <https://selfservice-practicum.d8case.ru>

Ingress IP на момент установки: 192.168.2.31. Пока DNS-записи не созданы, на рабочей станции нужны записи:

```
192.168.2.31 gitea-practicum.d8case.ru  
192.168.2.31 argocd-practicum.d8case.ru  
192.168.2.31 awx-practicum.d8case.ru  
192.168.2.31 selfservice-practicum.d8case.ru
```

Пароли не хранятся в Git. Получить созданные credentials можно так:

```
kubectl -n practicum-tks get secret practicum-gitea-admin-credentials  
kubectl -n practicum-tks get secret practicum-argocd-admin-credentials  
kubectl -n practicum-tks get secret practicum-awx-admin-password
```

Манифесты находятся в `manifests/practicum/`. Установка идиempотентна и не удаляет уже существующие на стенде объекты.

Актуальный контур `practicum-tks`

Диаграмма Mermaid преобразована в читаемую цепочку:

1	Разработчик: Git или Web
2	EnvironmentRequest в Gitea
3	practicum-request-controller
4	Generated desired state
5	Argo CD
6	Kubernetes app + DVP VM
7	AWX post-config и validation
8	Status в Git и portal
9	TTL
10	Удаление generated из Git
11	Prune только ресурсов environment

Разработчик: Git или Web → EnvironmentRequest в Gitea → practicum-request-controller → Generated desired state → Argo CD → Kubernetes app + DVP VM → AWX post-config и validation → Status в Git и portal → TTL → Удаление generated из Git → Prune только ресурсов environment

Argo CD Application practicum-demo читает gitops/environments/practicum. Все workload-объекты создаются только в namespace practicum-tks. Controller не выполняет kubectl delete: при истечении TTL он архивирует request и удаляет generated-каталог из Git, после чего Argo CD prune удаляет ресурсы с labels конкретного environment.

Ограничения self-service:

- максимум три активных окружения;
- максимум две одновременно работающие VM;
- пользователь выбирает только утверждённый профиль и TTL;
- VM используют VirtualImage, указанный в activeGoldenImage;
- AWX запускается после готовности guest OS и выполняет не более трёх попыток.

Golden image pipeline хранится в gitops/environments/practicum/golden-images. Git задаёт source URL, builder, versioned images и active-указатель. Проверены practicum-alpine-golden-3-23-v1 и practicum-alpine-golden-3-23-v2; active версия — v2. Старые версии не изменяются и доступны для rollback.

Расширенный DVP-контур

Application demo-platform синхронизирует путь:

```
gitops/environments/prod
```

Создать Application:

```
kubectl apply -f manifests/argocd/application-demo-platform.yaml
```

Проверить:

```
kubectl get application -n argocd demo-platform
kubectl get deploy,svc,ingress -n demo-prod
kubectl get vi,vd,vm -n demo-prod -o wide
kubectl get ns customer-a
```

Ожидаемое состояние DVP VM:

```
VirtualImage demo-alpine-cloud: Ready
VirtualDisk postgres-vm-root: Ready, 256Mi
VirtualMachine postgres-vm: Running
CPU: 1 core, coreFraction 5%
RAM: 512Mi
```

VM специально минимальная, чтобы стенд не потреблял лишние ресурсы.

Сценарии

Для действующего practicum-стенда используйте отдельный комплект:

- индекс и порядок показа (scenarios/d8case/README.md);
- подготовка демонстратора (scenarios/d8case/00-preflight.md);
- golden images (scenarios/d8case/02-golden-image-lifecycle.md);
- заявка через Git (scenarios/d8case/03-git-environment-request.md);
- заявка через Web (scenarios/d8case/04-web-self-service.md);
- административный lifecycle Victor (scenarios/d8case/05-victor-lifecycle.md);
- DVP VM drift correction (scenarios/d8case/06-vm-drift-correction.md).

Таблица ниже описывает исторический комплект и не является runbook для d8case.ru.

Сценарий	Что показывает
01. Initial Deploy (scenarios/01-initial-deploy.md)	Первичное развёртывание из Git.
02. Scale Application (scenarios/02-scale-application.md)	Scale через Git, а не <code>kubectl scale</code> .
03. Drift Correction (scenarios/03-drift-correction.md)	Self-healing после ручного drift.
04. VM Resize (scenarios/04-vm-resize.md)	Изменение параметров DVP VM через Git.
05. AWX Post-Configuration (scenarios/05-awx-post-config.md)	Post-config ОС/БД через AWX.
06. Broken Release And Rollback (scenarios/06-broken-release-and-rollback.md)	Ошибка image tag и rollback через Git.
07. Self-Service Tenant (scenarios/07-self-service-tenant.md)	Tenant onboarding через каталог в Git.
08. Golden Image Management (scenarios/08-golden-image-management.md)	Импорт исходного image из URL, builder VM, AWX customization и публикация golden image.
09. Self-Service Environment Request (scenarios/09-self-service-environment-request.md)	Разработчик выбирает профиль стенда через Git/request или web UI, Argo CD и AWX создают окружение.
10. Self-Service Portal (scenarios/10-self-service-portal.md)	Разработчик логинится через Dex, выбирает профиль и создаёт стенд через web portal.
11. Practicum End-to-End (scenarios/11-practicum-end-to-end.md)	Актуальный сценарий нового стенда: golden image, Git/Web request, AWX и TTL cleanup.
12. DVP VM Drift Correction (scenarios/12-dvp-vm-drift-correction.md)	Ручное изменение CPU/RAM DVP VM, Argo CD self-heal и безопасная модель RBAC.

Self-service UI

Для демонстрации developer-facing UX есть статическое web-приложение:

```
open self-service-ui/index.html
```

UI не создаёт ресурсы напрямую. Он генерирует EnvironmentRequest YAML и Git-команды. Далее request проходит через Git, review/merge, Argo CD sync и AWX post-configuration.

Для DVP VM self-service использует утверждённые ClusterVirtualImage, чтобы tenant namespace мог создавать диски из общего платформенного каталога образов без копирования namespaced VirtualImage.

GitOps/YAML-вариант описан в docs/self-service.ru.md (docs/self-service.ru.md) и scenarios/09-self-service-environment-request.md (scenarios/09-self-service-environment-request.md): разработчик создаёт EnvironmentRequest в gitops/self-service/requests/, а automation/controller должен сгенерировать manifests в gitops/self-service/generated/.

Self-service portal в кластере

Актуальный portal нового стенда размещён в practicum-tks и доступен по адресу:

```
https://selfservice-practicum.d8case.ru
```

Доступ закрыт через DKP DexAuthenticator. Portal создаёт только EnvironmentRequest; generated manifests создаёт отдельный controller. Portal показывает environment ID, фиксированный namespace practicum-tks, Git commit, Argo CD, приложение, DVP VM, фактический golden image, AWX job и TTL. Кнопка Выйти вызывает endpoint /logout, созданный DexAuthenticator, и завершает Dex-сессию перед входом под другим демонстрационным пользователем. Portal определяет канонического владельца по проверенному Dex e-mail и пересечению с разрешённой группой. Техническое значение X-Auth-Request-User не используется в имени EnvironmentRequest. Repository webhook Gitea отправляет push events на внутренний endpoint argocd-server.practicum-tks.svc.cluster.local/api/webhook. В allowlist Gitea разрешён только этот host, поэтому Argo CD обновляет Application сразу после commit, не ожидая периодического Git polling. Application и webhook используют одинаковый canonical repo URL `http://gitea-practicum.d8case.ru/...`; argocd-repo-server разрешает это имя локально через точечный host alias на ingress IP 192.168.2.31.

UI подробно объясняет профиль стенда, purpose, квоты и состав ресурсов. После создания заявки он показывает namespace, профиль, TTL, параметры приложения, service/ingress, VM/disk параметры и пути GitOps artifacts. После отправки UI показывает понятный пользовательский статус В работе, В очереди, Готово или Ошибка. Пока заявка обрабатывается, portal каждые 5 секунд обновляет текущий этап: Git/controller, Argo CD, Kubernetes, DVP, готовность гостевой ОС или AWX post-configuration. После трёх неуспешных AWX attempts заявка переходит в terminal state Error и показывает номер последнего job. Для Alpine playbook выбирает доступный versioned пакет PostgreSQL из репозитория образа вместо жёсткой привязки к одной версии.

Для профиля app-with-postgres-vm пользователь выбирает PostgreSQL 16, 17 или 18. Версия сохраняется в EnvironmentRequest, валидируется controller и передаётся в AWX как конкретный пакет postgresqlNN. Для любого VM-профиля portal показывает логин ansible, key-based способ аутентификации и готовую команду `d8 v ssh` с именем VM, namespace и локальным путём к ключу. Приватный ключ и пароль в Git/status не публикуются.

Ёмкость self-service задаётся переменными окружения request controller: MAX_ACTIVE_ENVIRONMENTS=7 и MAX_ACTIVE_VMS=7. При достижении одного из лимитов корректная заявка получает Queued/capacity-limit и автоматически продолжается после освобождения места.

Lifecycle-операции также проходят через GitOps. Пользователь управляет своими стендами в разделах Мои стенды и История, а Victor использует отдельный портал `https://vm-admin-practicum.d8case.ru`. Порталы создают EnvironmentAction в Git; request controller атомарно обновляет desired state, а Argo CD выполняет sync/prune. Прямой `kubectl delete` не используется.

Поддерживаются delete-environment, delete-vm, start-vm, stop-vm и restart-vm. Обычный пользователь может удалить только собственный стенд или VM с диском. Victor видит все tenant environments, обязан указать причину и не получает через портал доступ к golden builders или platform resources.

Для локального DNS добавьте:

```
192.168.2.31 selfservice-practicum.d8case.ru
```

Подробнее: docs/self-service-portal.ru.md (docs/self-service-portal.ru.md).

Что делает bootstrap

- Устанавливает Argo CD.
- Устанавливает Gitea.
- Создаёт пользователя и репозиторий в Gitea.
- Загружает проект в Gitea.
- Создаёт Application ansible-os-pods.
- Ждёт pod'ы ol-node-1 и ol-node-2.
- Устанавливает AWX operator и AWX.
- Создаёт AWX inventory, hosts, credential, project, execution environment и job template.
- Поднимает локальные port-forward'ы для UI.

Проверка pod-only сценария

```
kubectl get application -n argocd ansible-os-pods
kubectl get pods -n demo-os
kubectl exec -n demo-os deploy/ol-node-1 -- cat /etc/ansible-managed-by-awx
kubectl exec -n demo-os deploy/ol-node-2 -- cat /etc/ansible-managed-by-awx
```

Ожидаемый marker:

```
managed_by=AWX
deployed_by=Argo CD
host=...
kernel=...
```

Troubleshooting

run-demo-job.sh возвращает 401

Частая причина: localhost:3002 смотрит на старый AWX из другого kube-context. В DKP запускайте так:

```
AWX_URL=http://awx-demo.d8.kir.lab ./scripts/run-demo-job.sh
```

demo-platform OutOfSync

```
kubectl describe application -n argocd demo-platform
kubectl get events -n demo-prod --sort-by=.lastTimestamp
```

Если в ошибке есть data source cannot be changed if the VirtualDisk has already been provisioned, Argo CD пытается изменить immutable-источник уже подготовленного DVP VirtualDisk. Для demo-prod/postgres-vm-root в Application demo-platform специально настроен ignore /spec/dataSource и /spec/persistentVolumeClaim вместе с RespectIgnoreDifferences=true. Если нужно сменить базовый образ уже созданной VM, используйте новый диск или controlled restore/recreate flow, а не patch dataSource на существующем диске.

Если ошибка связана с gitops/self-service/generated/kustomization.yaml, проверьте, что файл содержит либо:

```
resources: []
```

либо многострочный список:

```
resources:  
  - dev-example
```

DVP disk долго в Provisioning

Проверить disk и events:

```
kubectl describe vd postgres-vm-root -n demo-prod  
kubectl get events -n demo-prod --sort-by=.lastTimestamp
```

Для первого запуска нормально, что image скачивается, а disk импортируется несколько минут.

AWX долго стартует

```
kubectl get job,pods -n awx  
kubectl logs -n awx job/awx-demo-migration-24.6.1 --tail=100
```

Rollback и очистка

Сценарные изменения откатывайте через Git:

```
git revert HEAD  
git push  
argocd app get demo-platform
```

Удаление локального стенда:

```
./scripts/destroy.sh
```

Для DVP-ресурсов дополнительно проверьте, что удаление VM/disk допустимо для текущего стенда:

```
kubectl get vi,vd,vm -n demo-prod
```

Паспорт стенда d8case.ru

docs/d8case/README.md

Этот раздел содержит актуальные параметры стенда practicum-tks.

Пошаговые демонстрации:

scenarios/d8case/README.md

Идентификаторы

Параметр	Значение
Kubernetes context	practicum-tks-api.d8case.ru
DKP Project	practicum-tks
Namespace	practicum-tks
Argo CD Application	practicum-demo
Gitea repository	practicum/practicum-demo
StorageClass	replicated
VirtualMachineClass	generic
Ingress IP	192.168.2.31

URL

Сервис	URL
Gitea	http://gitea-practicum.d8case.ru
Argo CD	http://argocd-practicum.d8case.ru
AWX	http://awx-practicum.d8case.ru
Portal разработчика	https://selfservice-practicum.d8case.ru
Portal Victor	https://vm-admin-practicum.d8case.ru

Платформенные объекты

Deployment/practicum-gitea
Deployment/argocd-server
Deployment/practicum-awx-web
Deployment/practicum-awx-task
Deployment/practicum-request-controller
Deployment/practicum-self-service-portal
Deployment/practicum-vm-admin-portal

Golden images

VirtualImage/practicum-alpine-base-3-23-v1
VirtualImage/practicum-alpine-golden-3-23-v1
VirtualImage/practicum-alpine-golden-3-23-v2
VirtualDisk/practicum-golden-builder-root
VirtualDisk/practicum-golden-builder-v2-root
VirtualMachine/practicum-golden-builder-vm
VirtualMachine/practicum-golden-builder-v2-vm
ConfigMap/practicum-golden-image-catalog

Active image читается командой:

```
kubectl get cm practicum-golden-image-catalog -n practicum-tks \
-o jsonpath='{.data.activeGoldenImage}{"\n"}'
```

Tenant environment

Environment ID всегда динамический:

```
practicum-env-<owner>-<purpose>-<suffix>
```

Не копируйте ID из старого протокола демонстрации. Получайте его из portal, Git request или labels:

```
kubectl get deploy -n practicum-tks \
-l demo.practicum/environment \
-o custom-columns='ENV:.metadata.labels.demo\practicum/environment,OWNER:.metadata.labels.dem
o\practicum/owner,READY:.status.readyReplicas'
```

Объекты environment:

```
Deployment/<environment-id>
Service/<environment-id>
Ingress/<environment-id>
VirtualDisk/<environment-id>-root
VirtualMachine/<environment-id>-vm
```

VM отсутствует у профиля app-only.

Пользователи

Пользователь	Группа	Назначение
Alice	practicum-payments-devs	app-only, app-with-vm
Boris	practicum-analytics-devs	app-only, PostgreSQL VM
Marina	practicum-qa-devs	все developer profiles
Victor	practicum-vm-operators	lifecycle tenant environments

Пароли не хранятся в Git.

Source of truth

```
gitops/environments/practicum/
gitops/self-service/practicum/requests/
gitops/self-service/practicum/actions/
gitops/self-service/practicum/status/
```

Историческая документация

Упоминания следующих значений относятся к предыдущему стенду:

```
d8.kir.lab
demo-prod
demo-platform
ansible-os-pods
postgres-vm
golden-builder-vm
```

Для текущего стенда заменять их механически нельзя: архитектура и набор сценариев изменились. Используйте материалы `scenarios/d8case/`.

Текущий статус

docs/STATUS.md

Обновлено: 2026-06-19 MSK

Правило работы

Перед изменениями читать docs/STATUS.md и docs/NEXT_STEPS.md. Архитектурные решения синхронно отражать в README.md и README.ru.md. После изменений обновлять этот файл.

Репозиторий

- локально: /Users/kir/code/ArgoAWXk8sDVPdemo;
- GitHub: kirka1206/ArgoAWXk8sDVPdemo;
- Gitea: practicum/practicum-demo;
- checkpoint до документационного коммита:
99e48b5a1548dc03dda31e85573d56a1bb9ac2cb;
- remote practicum-gitea не содержит пароль в URL;
- старые remote dkr-gitea и стенд d8.kir.lab не изменялись.

Новый стенд

- context: practicum-tks-api.d8case.ru;
- DKP Project и namespace: practicum-tks;
- ingress IP: 192.168.2.31;
- StorageClass: replicated;
- VirtualMachineClass: generic;
- квота: 4 CPU, 8Gi RAM, 30Gi storage.

Платформа Ready:

- Gitea 1.24.6;
- Argo CD 3.4.2;
- AWX Operator 2.19.1;
- AWX 24.6.1;
- Application practicum-demo: Synced/Healthy.

Рабочие Deployment платформы Ready: Gitea, все компоненты Argo CD, AWX Operator, AWX web/task, request controller и self-service portal. Deployment DexAuthenticator имеет состояние 2/2.

Пользователи и RBAC

- Alice: practicum-payments-devs, app-only и app-with-vm;
- Boris: practicum-analytics-devs, app-only и app-with-postgres-vm;
- Marina: practicum-qa-devs, все self-service профили;
- Victor: practicum-vm-operators, ограниченное управление DVP VM/дисками;
- Gitea bot: practicum-portal-bot;
- AWX account: practicum-automation;
- ServiceAccount: practicum-self-service-portal;

- ServiceAccount: practicum-request-controller.

Пароли хранятся только в ignored-файле local/practicum-demo-users.env.

Golden images

- source practicum-alpine-base-3-23-v1: Ready;
- immutable practicum-alpine-golden-3-23-v1: Ready;
- immutable practicum-alpine-golden-3-23-v2: Ready;
- active image: v2;
- AWX workflow job 40: successful;
- оба builder VM остановлены, ресурсы минимальные.
- сценарий scenarios/d8case/02-golden-image-lifecycle.md дополнен примерами альтернативных source images для демо: Alpine 3.23 v2, Alpine 3.22 v1 и Ubuntu 24.04 как более тяжёлый архитектурный пример;
- в сценарии явно описан rollback через отдельный Git commit, переключающий activeGoldenImage обратно на предыдущую версию; rollback влияет только на новые VM и не изменяет уже provisioned VirtualDisk.

Self-service

- portal: https://selfservice-practicum.d8case.ru;
- Certificate: Ready;
- DexAuthenticator: создан;
- Alice успешно вошла через Dex и видит только профили app-only и app-with-vm;
- в portal добавлена кнопка Выйти, использующая созданный DexAuthenticator endpoint /logout;
- первая заявка Alice выявила, что DKP Dex передаёт технический идентификатор в X-Auth-Request-User; controller отклонил её с owner is not approved;
- portal исправлен: owner определяется по allowlist e-mail и разрешённой группе, Environment ID нормализуется в lowercase, UI показывает причину отказа;
- progress UX добавлен: пользовательские статусы В работе, В очереди, Готово, Ошибка, текущий этап и polling каждые 5 секунд;
- повторная отправка блокируется до terminal state, ошибка связи не останавливает последующие проверки статуса;
- выявлен дефект профиля app-with-postgres-vm: AWX job 70 завершился failed, потому что Alpine 3.23 не содержит пакет postgresql15;
- playbook исправлен: выбирает самый новый доступный postgresqlNN;
- controller исправлен: после трёх failed AWX attempts выставляет Error с причиной вместо бесконечного Provisioning;
- исправление проверено на исходной заявке Marina practicum-env-marina-feature-10d265: повторный AWX job 73 завершился successful за 43.5 секунды, итоговый status заявки — Ready;
- portal и request contract расширены выбором PostgreSQL 16, 17 или 18; controller валидирует allowlist и передаёт точный пакет в AWX;

- для VM-профилей status и portal показывают пользователя ansible, SSH key authentication и готовую команду `d8 v ssh`; секреты в Git не добавляются;
 - обработчик временной ошибки Gitea сохраняет AWX job, attempts и runtime-поля существующего status, чтобы следующий reconcile не запускал job повторно;
 - request controller: Ready;
 - в Gitea создан repository push webhook на внутренний endpoint Argo CD;
 - Gitea webhook allowlist ограничен точным Service DNS `argocd-server.practicum-tks.svc.cluster.local`;
 - test delivery принят Argo CD: в логе `argocd-server` зафиксирован push event репозитория `practicum/practicum-demo`;
 - Application переведён на canonical Gitea URL из webhook payload;
 - только `argocd-repo-server` получил host alias `gitea-practicum.d8case.ru -> 192.168.2.31`; общекластерный DNS не изменялся;
 - реальный push проверен: Argo CD увидел новый revision за 9 секунд без ручного Sync или Refresh;
 - один namespace для всех environments: `practicum-tks`;
 - лимиты вынесены в env request controller:
`MAX_ACTIVE_ENVIRONMENTS=7, MAX_ACTIVE_VMS=7`;
 - изменение лимитов проверено на заявке `practicum-env-marina-bugfix-39884b`: после rollout controller она автоматически вышла из `Queued/capacity-limit` в `Provisioning`;
 - реализован GitOps lifecycle через EnvironmentAction: delete environment, delete VM+disk, start, stop и restart VM;
 - пользовательский portal получил Мои стенды, История и двойное подтверждение удаления собственных ресурсов;
 - добавлен отдельный Victor portal `vm-admin-practicum.d8case.ru`, доступный только `practicum-vm-operators`; причина административного действия обязательна;
 - action/request/generated/kustomization обновляются атомарным Gitea `repoChangeFiles commit`, фактическое удаление выполняет Argo CD `prune`;
 - controller не выполняет прямой `kubectl delete`;
 - AWX стартует после guest readiness, максимум 3 попытки.
- Live-приёмка lifecycle выполнена 2026-06-10:
- Victor portal пропускает только Victor с группой `practicum-vm-operators`; запрос с неверной группой получает 403;
 - пользовательский API не разрешает Alice удалить стенд Marina и возвращает 403;
 - `restart-vm` прошёл через Git action и versioned `VirtualMachineOperation`, после выполнения operation удалён из desired state, VM осталась Running;
 - `delete-vm` для `practicum-env-marina-bugfix-3b1e0b` удалил VM и `VirtualDisk` через Argo CD `prune`, сохранил Deployment, Service и Ingress и преобразовал request в app-only;
 - последующий `delete-environment` перевёл status через `DeletionRequested/Deleting` в `Cleaned`, архивировал request/action и удалил только app, Service и Ingress выбранного environment;

- Application practicum-demo после операций вернулся в Synced/Healthy;
- оба portal Deployment и request controller имеют 1/1 Ready, Certificate административного portal — Ready;
- desktop и mobile layout проверены на отсутствие горизонтального переполнения; фильтры, меню и модальные подтверждения проверены в браузере.

UX пользовательского portal дополнительно улучшен:

- карточка Мои стенды показывает те же основные данные, что и результат создания: Environment ID, namespace, owner, профиль, PostgreSQL, TTL, приложение, URL, VM/IP, ресурсы, golden image, SSH, AWX и Git commit;
- кнопка Обновить использует no-store, показывает время и количество полученных стендов, а ошибки загрузки выводит непосредственно на вкладке;
- чтение status-файлов из Gitea выполняется ограниченным параллельным пулом вместо последовательных HTTP-запросов, поэтому время обновления не растёт линейно вместе с историей environments;
- активная вкладка читает status только для текущих requests и выполняющихся actions; накопленный архив запрашивается отдельно при открытии Истории;
- во время provisioning и lifecycle-операций список автоматически обновляется каждые 5 секунд и объясняет текущий этап controller/Argo CD/DVP;
- поле Назначение снабжено динамическим пояснением: сейчас оно влияет на Environment ID и аудит, а ресурсы определяются выбранным профилем.

Административный portal Victor исправлен после алерта IngressResponses5xx:

- причиной были последовательный полный обход Git status каждые 5 секунд, наложение запросов и периодические ошибки/таймауты Gitea;
- backend получил 15-секундный cache, блокировку единственного refresh, загрузку status одним архивом ветки Gitea и возврат последнего успешного snapshot при временной ошибке Gitea;
- frontend не запускает новый refresh поверх незавершённого, polling увеличен до 15 секунд, ошибки показываются без очистки последней таблицы;
- кнопки Обзор, Стенды, Виртуальные машины, Операции и Аудит теперь являются настоящими представлениями с отдельной фильтрацией;
- API и браузер используют no-store, а UI показывает время последнего успешного обновления.
- pod template содержит декларативную версию ConfigMap-кода, поэтому изменение административного portal вызывает GitOps rollout Deployment, а не оставляет старый Python-процесс после состояния Synced.

Проверенное окружение:

- ID: practicum-env-alice-feature-019354;
- app: 1/1;
- VM: Running, 512Mi, 1 core / 5%, disk 768Mi;
- image: practicum-alpine-golden-3-23-v2;
- DVP AgentReady: True;
- AWX job 55: successful.

TTL проверен на `practicum-env-alice-demo-968c7a`: request архивирован, status `Cleaned`, Argo CD prune удалил только `app/ingress/VM/disk` этой заявки.

Окружение `practicum-env-alice-feature-019354` также штатно прошло TTL:

- request перенесён из `requests/` в `archive/`;
- `generated`-каталог удалён из `desired state`;
- status: `Cleaned`;
- `app`, `Service`, `Ingress`, `VirtualDisk` и `VirtualMachine` удалены Argo CD prune;
- namespace `practicum-tks` и платформенные компоненты сохранены.

Текущее состояние self-service:

- активных `EnvironmentRequest`: 0;
- активных `generated environments`: 0;
- работающих `tenant VM`: 0;
- `builder VM`: 2, обе `Stopped`;
- active image: `practicum-alpine-golden-3-23-v2`.

Отклонённая заявка с некорректным Dex owner удалена из активной очереди. Её status оставлен в Git как диагностический аудит.

Текущее Web-окружение Alice:

- ID: `practicum-env-alice-feature-537020`;
- профиль: `app-with-vm`;
- приложение: 1/1, Ingress 192.168.2.31;
- приложение опубликовано по HTTP; portal исправлен, чтобы не показывать несуществующий HTTPS для `tenant Ingress` без TLS;
- VM: `Running`, 10.66.0.32, `AgentReady=True`;
- SSH через `d8 v ssh` и локальный ключ проверен;
- AWX job 59 запущен автоматически и завершён `successful`.
- Git commit в status восстановлен: `507855ee9542e9a8b1c93b3b462c623429e1a32f`.

Второе Web-окружение Alice `practicum-env-alice-feature-fef235` имеет профиль `app-only` и штатно прошло TTL cleanup; виртуальная машина для него не создавалась. Окружение `practicum-env-alice-feature-537020` также очищено по TTL. Для выполнения сценария VM drift нужно создать новое временное окружение профиля `app-with-vm`.

Проверки

- Python syntax: пройдена;
- shell syntax: пройдена;
- Kustomize build: пройдена;
- server-side dry-run: пройден;
- Argo CD: `Synced/Healthy`;
- golden image Git -> Argo CD -> DVP -> AWX -> publish: пройден;
- Web request -> Git -> controller -> Argo CD -> DVP -> AWX: пройден;
- TTL cleanup: пройден.
- Python и встроенный JavaScript lifecycle portal: пройдены;
- atomic Gitea `repoChangeFiles commit`: пройден;
- user ownership и Victor group authorization: пройдены;
- `restart-vm`, `delete-vm`, `delete-environment`: пройдены;

- desktop/mobile UI lifecycle portal: пройден.

Сценарии

Добавлены сценарии:

- `scenarios/12-dvp-vm-drift-correction.md`: drift, self-heal и controlled restart;
- `scenarios/13-manual-environment-lifecycle.md`: пользовательское удаление собственных ресурсов и административные операции Victor через Git actions.

Для действующего стенда создан отдельный канонический комплект `scenarios/d8case/`:

- preflight с проверкой context, Application, DVP и URL;
- архитектурный рассказ Git → controller → Argo CD → DVP → AWX;
- golden image lifecycle с актуальными `practicum-*` объектами;
- Git/YAML self-service с `EnvironmentRequest`;
- Web self-service через Dex;
- административный lifecycle Victor;
- drift correction tenant VM;
- cleanup и troubleshooting.

В комплекте отсутствуют команды `demo-prod`, домен `d8.kir.lab` и имена объектов старого стенда. Корневые `scenarios/01-13` сохранены как исторические варианты и явно отделены в README.

PDF-документация

Сформирован навигационный PDF по проекту:

```
output/pdf/ArgoAWXk8sDVPdemo-project-guide.pdf
```

Источник сборки:

```
scripts/build_project_pdf.py
```

PDF включает README, паспорт `d8case.ru`, текущий статус, актуальные сценарии `scenarios/d8case`, исторические сценарии `01-13` и вспомогательные документы из `docs/`. В PDF есть оглавление, page numbers и bookmarks/outline для навигации. Выполнена визуальная проверка рендера выбранных страниц через PyMuPDF в `tmp/pdfs/project-guide-render/`.

PDF пересобран после проверки форматирования:

- fenced code blocks выводятся как код, без HTML-escape артефактов для кавычек и угловых скобок;
- Mermaid flowchart blocks преобразуются в читаемые PDF-диаграммы-цепочки, а не выводятся как сырой DSL;
- итоговый файл содержит 137 страниц и 69 bookmarks/outline entries;
- целевые страницы с архитектурной схемой и примером `EnvironmentRequest` визуально проверены через PyMuPDF.

Это стабильная точка продолжения: Application Synced/Healthy, временные environment-ресурсы очищены, golden images и остановленные builder-диски сохранены.

Известные особенности

- Gitea controller пишет status/generated коммитами в main; перед локальным push нужно сначала получить эти коммиты. Force-push запрещён.
- DNS для practicum hostnames должен указывать на 192.168.2.31; до появления DNS используются записи /etc/hosts.
- Self-signed сертификат portal требует доверия браузера либо замены issuer.
- ServiceAccount показываются с 0 token secrets — это штатное поведение новых Kubernetes, токены выдаются через projected service account volumes.

Актуальные сценарии d8case.ru: индекс

scenarios/d8case/README.md

Этот каталог является актуальным комплектом инструкций для стенда:

```
DKP Project: practicum-tks
Namespace:  practicum-tks
Context:    practicum-tks-api.d8case.ru
```

Материалы в корне scenarios/ сохраняются как история предыдущих стендов. Команды с demo-prod, d8.kir.lab, demo-platform, ansible-os-pods, golden-builder-vm и postgres-vm на новом стенде выполнять не следует.

Актуальные адреса

Компонент	Адрес
Gitea	http://gitea-practicum.d8case.ru
Argo CD	http://argocd-practicum.d8case.ru
AWX	http://awx-practicum.d8case.ru
Портал разработчика	https://selfservice-practicum.d8case.ru
Портал Victor	https://vm-admin-practicum.d8case.ru

Для временного разрешения имён:

```
192.168.2.31 gitea-practicum.d8case.ru
192.168.2.31 argocd-practicum.d8case.ru
192.168.2.31 awx-practicum.d8case.ru
192.168.2.31 selfservice-practicum.d8case.ru
192.168.2.31 vm-admin-practicum.d8case.ru
```

Tenant Ingress имеет динамическое имя:

```
192.168.2.31 <environment-id>.d8case.ru
```

Порядок показа

- Подготовка демонстратора (00-preflight.md).
- Архитектура и границы ответственности (01-architecture-tour.md).
- Управление golden images (02-golden-image-lifecycle.md).
- Self-service через Git и YAML (03-git-environment-request.md).
- Self-service через Web (04-web-self-service.md).
- Административное управление Victor (05-victor-lifecycle.md).
- Drift correction DVP VM (06-vm-drift-correction.md).
- Cleanup и troubleshooting (07-cleanup-and-troubleshooting.md).

Полный показ занимает 45-60 минут. Для короткого показа используйте сценарии 01, 02, 04 и 05.

Правила безопасности

- Не удалять namespace или DKP Project practicum-tks.
- Не изменять и не удалять опубликованные golden images v1 и v2.
- Не выполнять `kubectl delete` для tenant environments в штатном сценарии.

- Lifecycle выполнять через пользовательский или административный портал.
- Ручной drift показывать только на временной VM, которую можно пересоздать.
- Перед изменением Git выполнить `git fetch practicum-gitea main`.
- Никогда не использовать force-push: controller пишет status в main.

Соответствие старым сценариям

Старый сценарий	Статус на d8case.ru	Актуальная замена
01 Initial Deploy	Не применяется отдельно	00, 01
02 Scale Application	Не является основным сценарием	Не показывать
03 Drift Correction	Заменён DVP-вариантом	06
04 VM Resize	Входит в drift/GitOps объяснение	06
05 AWX Post-Config	Автоматизирован controller	03, 04
06 Broken Release	Не входит в текущий показ	Не показывать
07 Self-Service Tenant	Namespace не создаётся	03, 04
08 Golden Image	Полностью заменён	02
09 Git Environment Request	Полностью заменён	03
10 Self-Service Portal	Полностью заменён	04
11 Practicum End-to-End	Разделён на понятные этапы	01-05
12 DVP VM Drift	Актуализирован	06
13 Manual Lifecycle	Актуализирован	05

00. Подготовка демонстратора

scenarios/d8case/00-preflight.md

Цель

До начала показа подтвердить, что выбран правильный кластер, платформа работает, DNS разрешается, а в Git нет незавершённой операции.

1. Репозиторий

```
cd /Users/kir/code/ArgoAWXk8sDVPdemo
git status
git remote -v
git fetch practicum-gitea main
git log -1 --oneline practicum-gitea/main
```

Ожидается remote:

```
practicum-gitea http://gitea-practicum.d8case.ru/practicum/practicum-demo.git
```

Controller коммитит status в main, поэтому перед локальными изменениями:

```
git pull --rebase practicum-gitea main
```

Force-push запрещён.

2. Kubernetes context

```
kubectl config current-context
kubectl auth can-i get pods -n practicum-tks
kubectl get project practicum-tks 2>/dev/null || true
kubectl get namespace practicum-tks
```

Ожидаемый context:

```
practicum-tks-api.d8case.ru
```

3. Состояние платформы

```
kubectl get application -n practicum-tks practicum-demo

kubectl get deploy -n practicum-tks \
  practicum-gitea \
  argocd-server \
  practicum-awx-web \
  practicum-awx-task \
  practicum-request-controller \
  practicum-self-service-portal \
  practicum-vm-admin-portal
```

Ожидается:

- Application Synced/Healthy;
- каждый Deployment имеет READY 1/1.

4. Состояние DVP

```
kubectl get vi,vd,vm -n practicum-tks -o wide
kubectl get cm practicum-golden-image-catalog -n practicum-tks \
-o jsonpath='{.data.activeGoldenImage}{"\n"}'
```

Ожидается:

- source image practicum-alpine-base-3-23-v1 — Ready;
- golden images practicum-alpine-golden-3-23-v1 и v2 — Ready;
- active image — practicum-alpine-golden-3-23-v2;
- builder VM остановлены.

5. Проверка URL

```
for host in \
  gitea-practicum.d8case.ru \
  argocd-practicum.d8case.ru \
  awx-practicum.d8case.ru \
  selfservice-practicum.d8case.ru \
  vm-admin-practicum.d8case.ru; do
  printf '%-45s ' "$host"
  getent hosts "$host" 2>/dev/null || dscacheutil -q host -a name "$host"
done
```

Порталы используют self-signed certificate. Браузерное предупреждение можно принять для демонстрации.

6. Активные заявки

```
find gitops/self-service/practicum/requests \
-maxdepth 1 -type f ! -name README.md -print

kubectl get deploy,svc,ingress,vd,vm -n practicum-tks \
-l demo.practicum/environment
```

Не используйте старый Environment ID из документации. Каждый новый request получает динамический ID.

Что сказать аудитории

Мы сначала подтверждаем context и Application. На общем стенде особенно важно не перепутать namespace и не использовать команды старого демо. Все сегодняшние действия ограничены DKP Project и namespace practicum-tks.

01. Архитектура GitOps + DVP + AWX

scenarios/d8case/01-architecture-tour.md

Цель

До изменения ресурсов объяснить аудитории весь pipeline и показать, где настроена каждая связь.

Цепочка

Диаграмма Mermaid преобразована в читаемую цепочку:

1	Разработчик или Victor
2	Web portal или Git commit
3	Gitea practicum/practicum-demo
4	practicum-request-controller
5	Argo CD practicum-demo
6	Kubernetes и DVP
7	AWX Job Template
8	Гостевая ОС VM
9	Status в Git

Разработчик или Victor → Web portal или Git commit → Gitea practicum/practicum-demo → practicum-request-controller → Argo CD practicum-demo → Kubernetes и DVP → AWX Job Template → Гостевая ОС VM → Status в Git

1. Показать Git

Откройте:

```
http://gitea-practicum.d8case.ru/practicum/practicum-demo
```

Покажите:

```
gitops/environments/practicum/  
gitops/self-service/practicum/requests/  
gitops/self-service/practicum/status/  
gitops/self-service/practicum/actions/
```

Объяснение:

- requests — намерение пользователя;
- generated — desired state Kubernetes/DVP;
- actions — lifecycle-команды;
- status — результат controller, DVP, Argo CD и AWX.

2. Показать настройку Argo CD

Откройте:

```
http://argocd-practicum.d8case.ru
```

Application:

```
practicum-demo
```

CLI-проверка:

```
kubectl get application practicum-demo -n practicum-tks -o yaml |  
sed -n '/source:\/,/syncPolicy:p'
```

Покажите:

- repository URL Gitea;
- target revision main;
- path gitops/environments/practicum;
- automated sync, prune и self-heal.

Argo CD строит дерево не из README. Оно формируется из owner references, labels, namespace и фактически применённых Kubernetes-ресурсов.

3. Показать controller

```
kubectl get deploy practicum-request-controller -n practicum-tks  
kubectl logs deploy/practicum-request-controller -n practicum-tks --tail=50
```

Controller:

- читает EnvironmentRequest;
- проверяет owner, group, profile, TTL и лимиты;
- создаёт generated manifests через атомарный commit;
- ждёт готовности приложения и VM;
- запускает AWX;
- записывает status обратно в Git.

Это не controller Argo CD. Это отдельный компонент данного демо-проекта.

4. Показать AWX

Откройте:

```
http://awx-practicum.d8case.ru
```

Покажите:

- Project с репозиторием practicum-demo;
- Job Template Practicum Environment Post-Config;
- Workflow/Jobs golden image;
- inventory host, создаваемый или обновляемый для VM.

Argo CD не выполняет Ansible. AWX запускает playbook, а controller вызывает AWX API и затем читает статус job.

5. Показать DVP

В Web DKP откройте Project practicum-tks, затем виртуальные машины и образы.

```
kubect1 get vi,vd,vm,vmp -n practicum-tks -o wide
```

Ожидаемый вывод рассказа

Git хранит намерение и desired state. Controller реализует бизнес-процесс.

Argo CD синхронизирует Kubernetes/DVP-объекты. DVP исполняет виртуализацию.

AWX меняет состояние внутри гостевой ОС. Portal только создаёт запрос и показывает status, но не удаляет ресурсы напрямую.

02. Управление golden images

scenarios/d8case/02-golden-image-lifecycle.md

Цель

Показать цепочку:

```
URL source image в Git
→ Argo CD
→ DVP VirtualImage
→ builder disk и VM
→ AWX prepare/validate/shutdown
→ versioned golden image
→ activeGoldenImage
→ новые tenant VM
```

Исходное состояние

```
export NAMESPACE=practicum-tks
export APP_NAME=practicum-demo

kubectl get application "$APP_NAME" -n "$NAMESPACE"
kubectl get vi,vd,vm -n "$NAMESPACE" -o wide
```

1. Источник образа

В Gitea откройте:

```
gitops/environments/practicum/golden-images/source-image.yaml
```

Текущий URL:

```
https://dl-cdn.alpinelinux.org/alpine/v3.23/releases/cloud/generic_alpine-3.23.3-x86_64-bios-cloudinit-r0.qcow2
```

Альтернативные source images для демонстрации

Для live-показа лучше использовать небольшие cloud images. Самый безопасный вариант для текущего стенда — Alpine: импорт быстрый, размер небольшой, AWX playbooks уже адаптированы.

Пример новой версии на той же ветке Alpine:

```
apiVersion: virtualization.deckhouse.io/v1alpha2
kind: VirtualImage
metadata:
  name: practicum-alpine-base-3-23-v2
  namespace: practicum-tks
spec:
  storage: ContainerRegistry
  dataSource:
    type: HTTP
    http:
      url: https://dl-cdn.alpinelinux.org/alpine/v3.23/releases/cloud/generic_alpine-3.23.3-x86_64-bios-cloudinit-r0.qcow2
```

Пример смены minor-линейки Alpine:

```

apiVersion: virtualization.deckhouse.io/v1alpha2
kind: VirtualImage
metadata:
  name: practicum-alpine-base-3-22-v1
  namespace: practicum-tks
spec:
  storage: ContainerRegistry
  dataSource:
    type: HTTP
    http:
      url: https://dl-cdn.alpinelinux.org/alpine/v3.22/releases/cloud/generic_alpine-3.22.2-x86_64-bios-cloudinit-r0.qcow2

```

Более понятный аудитории, но более тяжёлый вариант — Ubuntu cloud image:

```

apiVersion: virtualization.deckhouse.io/v1alpha2
kind: VirtualImage
metadata:
  name: practicum-ubuntu-base-24-04-v1
  namespace: practicum-tks
spec:
  storage: ContainerRegistry
  dataSource:
    type: HTTP
    http:
      url: https://cloud-images.ubuntu.com/noble/current/noble-server-cloudimg-amd64.img

```

Для короткой демонстрации рекомендуется показывать переход:

Alpine 3.22 → Alpine 3.23

или:

Alpine 3.23 v1 → Alpine 3.23 v2

Ubuntu лучше оставить как архитектурный пример: он крупнее, дольше импортируется и может потребовать отдельной адаптации AWX playbooks.

Проверка:

```

kubectl get vi practicum-alpine-base-3-23-v1 \
  -n "$NAMESPACE" -o wide

```

Что сказать:

Администратор задаёт URL и версию в Git. DVP импортирует образ, а не использует локальный файл с ноутбука.

2. Builder v1 и v2

```

kubectl get vd \
  practicum-golden-builder-root \
  practicum-golden-builder-v2-root \
  -n "$NAMESPACE" -o wide

kubectl get vm \
  practicum-golden-builder-vm \
  practicum-golden-builder-v2-vm \
  -n "$NAMESPACE" -o wide

```

Ожидается:

- 1 core, 5%, 512Mi;
- VM Stopped;
- disks Ready, InUse=False.

Builder VM не запускаются для обычной демонстрации: golden images v1/v2 уже опубликованы. Это экономит ресурсы общего стенда.

3. AWX pipeline

В AWX откройте успешный golden image workflow и покажите этапы:

- prepare;
- customization;
- validation;
- shutdown;
- publication.

Playbooks находятся в:

```
gitops/awx/playbooks/prepare-golden-image.yml
gitops/awx/playbooks/validate-golden-image.yml
```

4. Опубликованные версии

```
kubectl get vi \
  practicum-alpine-golden-3-23-v1 \
  practicum-alpine-golden-3-23-v2 \
  -n "$NAMESPACE" -o wide
```

Обе версии должны оставаться Ready. Новая версия создаётся новым объектом, а не изменением data source уже provisioned image/disk.

5. Active image

В Gitea откройте:

```
gitops/environments/practicum/golden-images/catalog.yaml
```

Проверка:

```
kubectl get cm practicum-golden-image-catalog -n "$NAMESPACE" \
  -o jsonpath='{.data.activeGoldenImage}'
```

Ожидается:

```
practicum-alpine-golden-3-23-v2
```

Controller использует это значение при генерации новой tenant VM.

6. Как показать выпуск v3

Не изменяйте v1/v2. Для реального выпуска:

- создать новые VirtualDisk и builder VM с суффиксом v3;
- запустить AWX prepare и validation;
- опубликовать practicum-alpine-golden-3-23-v3;
- отдельным commit изменить activeGoldenImage;
- создать новый environment и показать, что его диск использует v3.

Rollback

Rollback не удаляет v2. Отдельным Git commit верните:

```
activeGoldenImage: practicum-alpine-golden-3-23-v1
```

Полный пример:

```
git fetch practicum-gitea main
git pull --rebase practicum-gitea main

$EDITOR gitops/environments/practicum/golden-images/catalog.yaml
git diff
git add gitops/environments/practicum/golden-images/catalog.yaml
git commit -m "Rollback active golden image to v1"
git push practicum-gitea main
```

После синхронизации:

```
kubectl get cm practicum-golden-image-catalog -n "$NAMESPACE" \
-o jsonpath='{.data.activeGoldenImage}'
```

Ожидаемо:

```
practicum-alpine-golden-3-23-v1
```

Это влияет только на новые VM. Уже созданные VM сохраняют исходный диск. Такой rollback не пересобирает и не меняет существующие VirtualDisk: это важное свойство безопасной эксплуатации. Если нужно перевести уже работающий стенд на другой образ, создавайте новый environment или отдельный controlled recreate/restore flow.

Нельзя делать

```
# Не выполнять:
kubectl patch vd <provisioned-disk> ...
kubectl delete vi practicum-alpine-golden-3-23-v1 ...
```

У provisioned VirtualDisk нельзя менять data source. Для новой версии нужен новый versioned объект.

03. Self-service через Git и YAML

scenarios/d8case/03-git-environment-request.md

Цель

Показать, что разработчик коммитит только декларативную заявку, а controller, Argo CD, DVP и AWX создают стенд без ручного `kubectl apply`.

Все ресурсы создаются в namespace `practicum-tks`. Новый DKP Project или namespace для каждой заявки не создаётся.

1. Подготовить переменные

```
cd /Users/kir/code/ArgoAWXk8sDVPdemo
git fetch practicum-gitea main
git pull --rebase practicum-gitea main

export ENV_ID="practicum-env-marina-demo-$(date +%H%M%S)"
export CREATED_AT=$(date -u +%Y-%m-%dT%H:%M:%SZ)"
echo "$ENV_ID $CREATED_AT"
```

Имя должно:

- начинаться с `practicum-env-`;
- содержать только lowercase, цифры и `-`;
- быть не длиннее 63 символов.

2. Создать EnvironmentRequest

```
cat > "gitops/self-service/practicum/requests/${ENV_ID}.yaml" <<EOF
{
  "apiVersion": "demo.practicum/v1",
  "kind": "EnvironmentRequest",
  "metadata": {
    "name": "${ENV_ID}"
  },
  "spec": {
    "owner": "marina-volkova-practicum",
    "email": "marina.volkova.practicum@demo.local",
    "groups": ["practicum-qa-devs"],
    "profile": "app-with-vm",
    "purpose": "demo",
    "ttl": "2h",
    "createdAt": "${CREATED_AT}"
  }
}
EOF
```

Разрешённые профили:

Profile	Результат
app-only	Deployment, Service, Ingress
app-with-vm	приложение и минимальная Linux VM
app-with-postgres-vm	приложение, VM и PostgreSQL post-config

Для PostgreSQL добавьте:

```
"postgresql": {"version": "18"}
```

Поддерживаются версии 16, 17, 18.

3. Commit и push

```
git add "gitops/self-service/practicum/requests/${ENV_ID}.yaml"
git commit -m "Request practicum environment ${ENV_ID}"
git push practicum-gitea main
```

GitHub является копией проекта, но live Argo CD читает Gitea.

4. Наблюдать Git

В Gitea покажите появление:

```
gitops/self-service/practicum/requests/<environment-id>.yaml
gitops/environments/practicum/self-service/generated/<environment-id>/
gitops/self-service/practicum/status/<environment-id>.json
```

Controller может создать несколько status-коммитов. Это штатно.

5. Наблюдать Argo CD

```
watch -n 2 kubectl get application practicum-demo -n practicum-tks
```

В UI Argo CD найдите ресурсы по Environment ID.

6. Наблюдать Kubernetes/DVP

```
kubectl get deploy,svc,ingress,vd,vm -n practicum-tks \
-l "demo.practicum/environment=${ENV_ID}" -w
```

Для VM:

```
kubectl get vm "${ENV_ID}-vm" -n practicum-tks -o wide
```

7. Наблюдать AWX

В AWX откройте Job Template:

```
Practicum Environment Post-Config
```

Controller запустит job после готовности VM и guest agent.

8. Прочитать итоговый status

```
git pull --rebase practicum-gitea main
jq . "gitops/self-service/practicum/status/${ENV_ID}.json"
```

Ожидается:

- state: Ready;
- приложение ready 1/1;
- VM Running;

- IP VM;
- AWX successful;
- команда `d8 v ssh`.

9. Доступ

```
kubectl get ingress "${ENV_ID}" -n practicum-tks  
  
d8 v ssh "ansible@${ENV_ID}-vm" \  
  --namespace practicum-tks \  
  --identity-file local/practicum-ssh/id_ed25519 \  
  --local-ssh
```

Cleanup

Удаляйте стенд через портал пользователя или Victor. Не удаляйте request и generated-каталог вручную во время демонстрации: lifecycle должен остаться аудируемым.

04. Self-service через Web

scenarios/d8case/04-web-self-service.md

Цель

Показать удобный интерфейс разработчика, который создаёт тот же `EnvironmentRequest`, что и ручной Git-сценарий.

Пользователи и профили

Пользователь	Группа	Профили
Alice	practicum-payments-devs	app-only, app-with-vm
Boris	practicum-analytics-devs	app-only, app-with-postgres-vm
Marina	practicum-qa-devs	все профили

Для полного показа используйте Marina:

```
marina.volkova.practicum@demo.local
```

Пароль берётся из локального `ignored`-файла, не из Git.

1. Открыть портал

```
https://selfservice-practicum.d8case.ru
```

Примите предупреждение self-signed certificate и войдите через Dex.

2. Объяснить поля

Профиль стенда

Профиль определяет ресурсы:

- Контейнерное приложение: Deployment, Service, Ingress;
- Приложение + Linux VM: добавляются DVP disk/VM и AWX bootstrap;
- Приложение + PostgreSQL VM: добавляется выбранная версия PostgreSQL.

Пользователь не задаёт произвольные CPU/RAM/image.

Назначение

Назначение:

- входит в Environment ID;
- сохраняется в Git и аудите;
- не меняет ресурсы профиля.

TTL

TTL определяет автоматическое время жизни. После истечения controller меняет desired state в Git, а Argo CD prune удаляет только ресурсы стенда.

3. Создать стенд

Для наиболее наглядного показа:

```
Профиль: Приложение + PostgreSQL VM
PostgreSQL: 18
Назначение: Демонстрация
TTL: 4h
```

Нажмите [Отправить заявку](#).

4. Читать прогресс

Portal обновляет status каждые 5 секунд. Объясните этапы:

- Submitted — request записан в Gitea;
- Provisioning — controller создал desired state;
- Argo CD создаёт Deployment, Service, Ingress, disk и VM;
- DVP запускает VM;
- AWX выполняет post-config;
- Ready — все проверки завершены.

5. Проверить результат

Portal должен показать:

- Environment ID;
- namespace practicum-tks;
- owner и profile;
- PostgreSQL version;
- TTL;
- приложение ready 1/1;
- URL;
- VM, IP, CPU/RAM/disk и image;
- AWX job/status;
- готовую команду `d8 v ssh`.

В терминале:

```
export ENV_ID=<скопировать-из-портала>

kubectrl get deploy,svc,ingress,vd,vm -n practicum-tks \
  -l "demo.practicum/environment=${ENV_ID}" -o wide
```

6. Показать Git

В Gitea найдите Environment ID в каталогах:

```
gitops/self-service/practicum/requests/
gitops/environments/practicum/self-service/generated/
gitops/self-service/practicum/status/
```

Главная мысль:

Portal не является вторым control plane. Он создаёт Git artifact, а дальше работает та же цепочка controller → Argo CD → DVP → AWX.

7. Мои стенды

Откройте вкладку Мои стенды:

- видны только environments текущего пользователя;
- кнопка Обновить читает актуальный Git status;
- доступны удаление VM и полного стенда;
- для выполняющейся операции показывается текущий этап.

Ожидаемый результат

Пользователь получает готовый временный стенд без доступа на создание Kubernetes/DVP-объектов и без ручного обращения к администратору.

05. Административный lifecycle Victor

scenarios/d8case/05-victor-lifecycle.md

Цель

Показать, как администратор управляет чужими tenant environments без нарушения GitOps и без прямого `kubectl delete`.

Исходное состояние

- существует временный environment Alice или Marina;
- Application practicum-demo — Synced/Healthy;
- Victor входит через группу practicum-vm-operators.

1. Открыть портал

```
https://vm-admin-practicum.d8case.ru
```

Пользователь:

```
victor.melnikov.practicum@demo.local
```

2. Показать представления

- Обзор: KPI по всем tenant environments;
- Стенды: активные стенды;
- Виртуальные машины: только environments с VM;
- Операции: lifecycle actions;
- Аудит: очищенные стенды и последние действия.

Golden builder VM не отображаются, потому что портал читает только status tenant environments с префиксом `practicum-env-`.

3. Выбрать стенд

Используйте поиск по owner:

```
alice-koroleva-practicum
```

Скопируйте Environment ID:

```
export ENV_ID=<environment-id>
export VM_NAME="${ENV_ID}-vm"
```

4. Stop/Start/Restart

На стенде с VM:

- нажмите Stop;

- укажите причину;
- подтвердите;
- наблюдайте status и Argo CD;
- повторите Start и Restart.

Проверка:

```
kubectl get vm "$VM_NAME" -n practicum-tks -w
kubectl get vmop -n practicum-tks \
  -l "demo.practicum/environment=${ENV_ID}"
```

Portal создаёт EnvironmentAction. Controller создаёт versioned VirtualMachineOperation. После завершения operation удаляется из desired state, а action архивируется.

5. Удалить только VM

Выберите Удалить VM, укажите причину и подтвердите.

Ожидается:

- VM и VirtualDisk удаляются Argo CD prune;
- Deployment, Service и Ingress сохраняются;
- request преобразуется в app-only;
- итоговый status — Ready, отметка VM removed.

```
kubectl get deploy,svc,ingress,vd,vm -n practicum-tks \
  -l "demo.practicum/environment=${ENV_ID}"
```

6. Удалить стенд целиком

Выберите Удалить стенд, укажите причину и подтвердите.

Статусы:

```
DeletionRequested → Deleting → Cleaned
```

Controller:

- отменяет активный AWX job, если он выполняется;
- архивирует request;
- удаляет generated desired state;
- Argo CD prune удаляет ресурсы Environment ID;
- сохраняет actor, reason, commit и outcome.

7. Проверить аудит

```
git pull --rebase practicum-gitea main

ls gitops/self-service/practicum/actions-archive |
  grep "${ENV_ID}"

jq . "gitops/self-service/practicum/status/${ENV_ID}.json"
```

Что нельзя делать Victor

Не удаляйте GitOps VM через Web DKP/DVP. Если ресурс остаётся в Git, Argo CD может восстановить его как drift. Для штатного lifecycle используйте административный портал.

Не удаляется:

- DKP Project practicum-tks;
- namespace practicum-tks;
- Gitea, Argo CD, AWX и порталы;
- golden images и builder VM;
- environments других пользователей.

06. Drift correction DVP VM

scenarios/d8case/06-vm-drift-correction.md

Цель

Показать, что ручное изменение GitOps-managed VM обнаруживается Argo CD и возвращается к desired state из Git.

Важно

Сценарий выполняется только на временной tenant VM. Не используйте golden builder VM.

1. Выбрать VM

```
kubectl get vm -n practicum-tks \
  -l demo.practicum/environment \
  -o custom-columns='VM:.metadata.name,ENV:.metadata.labels.demo\practicum/environment,OWNER:.metadata.labels.demo\practicum/owner,CPU:.spec.cpu.cores,FRACTION:.spec.cpu.coreFraction,MEMORY:.spec.memory.size,PHASE:.status.phase'
```

```
export VM_NAME=<tenant-vm>
export NAMESPACE=practicum-tks
export APP_NAME=practicum-demo
```

Проверьте label:

```
kubectl get vm "$VM_NAME" -n "$NAMESPACE" \
  -o jsonpath='{.metadata.labels.demo\practicum/environment}'
```

2. Зафиксировать desired параметры

```
kubectl get vm "$VM_NAME" -n "$NAMESPACE" \
  -o jsonpath='{.spec.cpu.cores} fraction={.spec.cpu.coreFraction} memory={.spec.memory.size}'
```

Обычно tenant VM имеет:

```
cores=1 fraction=5% memory=512Mi
```

3. Открыть наблюдение

```
kubectl get application "$APP_NAME" -n "$NAMESPACE" -w
```

В Argo CD откройте Application practicum-demo и выбранную VM.

4. Создать drift

Измените только memory:

```
kubectl patch vm "$VM_NAME" -n "$NAMESPACE" \
  --type merge \
  -p '{"spec":{"memory":{"size":"640Mi"}}}'
```

5. Показать self-heal

```
watch -n 1 kubectl get vm "$VM_NAME" -n "$NAMESPACE" \
-o custom-columns='NAME:.metadata.name,MEMORY:.spec.memory.size,PHASE:.status.phase,RESTART:.status.restartAwaitingChanges'
```

Ожидаемая цепочка:

```
live memory = 640Mi
→ Application OutOfSync
→ automated selfHeal
→ memory снова 512Mi
→ Application Synced/Healthy
```

OutOfSync может быть очень коротким.

6. Controlled restart

Если DVP показывает ожидающие применения изменения, restart выполняйте через портал Victor, а не прямой DVP-командой. Тогда action и причина сохраняются в Git-аудите.

Граница ответственности AWX

В этом сценарии AWX не запускается. CPU/RAM — декларативная инфраструктурная конфигурация DVP. AWX нужен для изменений внутри гостевой ОС.

Опционально: ручное удаление VM

Этот блок нужен только для объяснения риска, а не для основного показа:

```
kubectl delete vm "$VM_NAME" -n "$NAMESPACE"
```

Argo CD пересоздаст VM, пока она остаётся в Git. Удаление диска может привести к потере данных. Поэтому штатное удаление выполняется через EnvironmentAction в портале Victor.

07. Cleanup и troubleshooting

scenarios/d8case/07-cleanup-and-troubleshooting.md

Штатный cleanup

Пользователь

```
selfservice-practicum.d8case.ru  
→ Мои стенды  
→ Удалить стенд
```

Пользователь может удалить только свой environment.

Victor

```
vm-admin-practicum.d8case.ru  
→ Стенды  
→ выбрать Environment ID  
→ Удалить стенд  
→ указать причину
```

Проверка cleanup

```
export ENV_ID=<environment-id>  
  
kubectl get deploy,svc,ingress,vd,vm,vmap -n practicum-tks \\\n  -l "demo.practicum/environment=${ENV_ID}"  
  
git pull --rebase practicum-gitea main  
jq . "gitops/self-service/practicum/status/${ENV_ID}.json"
```

Ожидается:

- Kubernetes/DVP-ресурсов Environment ID нет;
- status Cleaned;
- request находится в archive;
- action находится в actions-archive;
- Application Synced/Healthy.

Не удалять

```
namespace/practicum-tks  
VirtualImage/practicum-alpine-golden-3-23-v1  
VirtualImage/practicum-alpine-golden-3-23-v2  
VirtualMachine/practicum-golden-builder-vm  
VirtualMachine/practicum-golden-builder-v2-vm
```

Диагностика Argo CD

```
kubectl get application practicum-demo -n practicum-tks  
kubectl describe application practicum-demo -n practicum-tks  
kubectl logs deploy/argocd-application-controller -n practicum-tks --tail=100  
kubectl logs deploy/argocd-repo-server -n practicum-tks --tail=100
```


Если provisioned VirtualDisk отклоняет изменение data source, не patch'ите его. Создайте новый versioned disk/image.

Диагностика controller

```
kubectl get deploy practicum-request-controller -n practicum-tks
kubectl logs deploy/practicum-request-controller -n practicum-tks --tail=200
```

capacity-limit означает достижение:

```
MAX_ACTIVE_ENVIRONMENTS=7
MAX_ACTIVE_VMS=7
```

Лимиты задаются env-переменными Deployment controller.

Диагностика AWX

```
kubectl get pods -n practicum-tks | grep practicum-awx
kubectl logs deploy/practicum-awx-task -n practicum-tks --tail=100
```

В UI AWX откройте Job ID из status environment.

Диагностика порталов

```
kubectl get deploy,svc,ingress,dexauthenticator,certificate \
-n practicum-tks | grep -E 'selfservice|vm-admin'

kubectl logs deploy/practicum-self-service-portal \
-n practicum-tks --tail=100

kubectl logs deploy/practicum-vm-admin-portal \
-n practicum-tks --tail=100
```

При IngressResponses5xx проверьте backend logs и /api/environments. Административный portal использует cache и один архивный запрос к Gitea.

Финальная проверка стенда

```
kubectl get application practicum-demo -n practicum-tks
kubectl get vi,vd,vm -n practicum-tks -o wide
kubectl get deploy -n practicum-tks
git status
```

Исторические сценарии: 01 Initial Deploy

scenarios/01-initial-deploy.md

Цель

Показать первичное развёртывание приложения, инфраструктурных объектов и базовой VM-модели из Git.

Исходное состояние

- Репозиторий подключен к ArgoCD Application demo-platform.
- В кластере есть ArgoCD и доступ к Git-репозиторию.
- Для DVP VM используется реальный минимальный manifest `gitops/environments/prod/dvp-postgres-vm.yaml`.

Что меняем в Git

Добавляем или обновляем initial manifests:

- `gitops/environments/prod/namespace.yaml`
- `gitops/environments/prod/rbac.yaml`
- `gitops/environments/prod/demo-app.yaml`
- `gitops/environments/prod/monitoring.yaml`
- `gitops/environments/prod/values.yaml`
- `gitops/environments/prod/dvp-postgres-vm.yaml`

Пошаговое выполнение

```
argocd app get demo-platform
git add .
git commit -m "Initial demo environment"
git push
argocd app get demo-platform
kubectl get ns demo-prod
kubectl get deploy,svc,ingress -n demo-prod
kubectl get vm -n demo-prod
kubectl get events -n demo-prod --sort-by=.lastTimestamp
```

В DKP/DVP стенде команда `kubectl get vm` должна показать `postgres-vm` в namespace `demo-prod`.

Что показывать в ArgoCD

- Application demo-platform.
- Status Synced и Healthy.
- Дерево ресурсов: namespace, RBAC, Deployment, Service, Ingress, DVP VirtualImage, VirtualDisk, VirtualMachine, заготовка monitoring-объектов.
- Sync waves: сначала базовые объекты, затем приложение и monitoring.

Что показывать в AWX

На этом сценарии AWX ещё не является главным действующим лицом. Достаточно показать, что playbooks и hooks уже лежат в Git и будут использованы на следующих шагах.

Проверка через kubectl

```
kubectl get ns demo-prod
kubectl get deploy demo-app -n demo-prod
kubectl get svc,ingress -n demo-prod
kubectl get configmap demo-monitoring-rules -n demo-prod
```

Ожидаемый результат

- ArgoCD Application находится в состоянии Synced/Healthy.
- demo-app запущен.
- Ingress и Service созданы.
- Все изменения пришли из Git, без ручного kubectl apply.

Rollback

```
git revert HEAD
git push
argocd app get demo-platform
```

Пояснение для демонстратора

На этом шаге важно подчеркнуть, что оператор не меняет инфраструктуру напрямую. Он меняет декларативное описание в Git, после чего ArgoCD приводит кластер к требуемому состоянию. Это снижает риск ручных ошибок и делает изменения воспроизводимыми.

Исторические сценарии: 02 Scale Application

scenarios/02-scale-application.md

Цель

Показать масштабирование приложения через изменение Git, а не через ручную команду `kubectl scale`.

Исходное состояние

- `demo-platform` находится в Synced/Healthy.
- В `gitops/environments/prod/values.yaml` указано `app.replicas: 2`.
- В live manifest `gitops/environments/prod/demo-app.yaml` у Deployment указано `replicas: 2`.

Что меняем в Git

Файл:

```
gitops/environments/prod/values.yaml
```

Было:

```
app:
  replicas: 2
```

Стало:

```
app:
  replicas: 4
```

Для текущего `kustomize`-примера также измените `spec.replicas` в `gitops/environments/prod/demo-app.yaml` с 2 на 4.

Пошаговое выполнение

```
git diff
git add gitops/environments/prod/values.yaml gitops/environments/prod/demo-app.yaml
git commit -m "Scale demo app to 4 replicas"
git push
argocd app get demo-platform
kubectl get deploy demo-app -n demo-prod
kubectl get pods -n demo-prod -l app=demo-app
```

Что показывать в ArgoCD

- Diff по Deployment `demo-app`.
- Автоматическую синхронизацию.
- Переход Application обратно в Synced/Healthy.

Что показывать в AWX

AWX здесь не запускается. Этот сценарий специально показывает границу ответственности: ArgoCD меняет Kubernetes workload, AWX нужен для процедурной настройки ОС или VM.

Проверка через kubectl

```
kubectl get deploy demo-app -n demo-prod
kubectl get pods -n demo-prod -l app=demo-app
```

Ожидаемый результат

- replicas = 4.
- Количество pod увеличилось.
- Ручных изменений в Kubernetes не было.

Rollback

```
git revert HEAD
git push
argocd app get demo-platform
kubectl get deploy demo-app -n demo-prod
```

Пояснение для демонстратора

Здесь стоит проговорить, что масштабирование становится обычным изменением в Git. Это удобно для production-процессов: есть review, история изменений, понятный rollback и одинаковый механизм доставки для приложений и инфраструктуры.

Исторические сценарии: 03 Drift Correction

scenarios/03-drift-correction.md

Цель

Показать self-healing и устранение configuration drift.

Исходное состояние

- В Git для demo-app задано ожидаемое количество реплик.
- В ArgoCD Application demo-platform включен selfHeal: true.

Что меняем в Git

В Git ничего не меняем. Специально ломаем live-состояние в Kubernetes:

```
kubectl scale deployment demo-app --replicas=10 -n demo-prod
```

Пошаговое выполнение

```
kubectl get deploy demo-app -n demo-prod
kubectl scale deployment demo-app --replicas=10 -n demo-prod
kubectl get deploy demo-app -n demo-prod
argocd app get demo-platform
kubectl get deploy demo-app -n demo-prod
```

Что показывать в ArgoCD

- Временный OutOfSync или diff по replicas.
- Автоматический self-heal.
- Возврат в Synced/Healthy.

Что показывать в AWX

AWX не участвует. Это чистый GitOps-сценарий про контроль desired state на Kubernetes API.

Проверка через kubectl

```
kubectl get deployment demo-app -n demo-prod
kubectl get pods -n demo-prod -l app=demo-app
```

Ожидаемый результат

- Ручное изменение откатилось.
- replicas снова равно значению из Git.
- ArgoCD показывает Synced/Healthy.

Rollback

Rollback не нужен: Git не менялся. Если self-heal отключен, выполните ручной sync:

```
argocd app sync demo-platform
```

Пояснение для демонстратора

Это сильный момент для архитектурной аудиторией: Git остаётся источником истины даже если кто-то обошел процесс и поменял кластер вручную. ArgoCD обнаруживает расхождение и возвращает систему в описанное состояние.

Исторические сценарии: 04 VM Resize

scenarios/04-vm-resize.md

Цель

Показать изменение параметров виртуальной машины через Git.

Исходное состояние

- В Git есть gitops/environments/prod/values.yaml.
- В Git есть реальный минимальный DVP manifest gitops/environments/prod/dvp-postgres-vm.yaml.
- VM уже развернута с минимальными ресурсами для стенда: 1 core, coreFraction: 5%, 512Mi RAM, disk 256Mi.

Что меняем в Git

Файл:

```
gitops/environments/prod/values.yaml
```

Было:

```
vm:
  cpu: 1
  coreFraction: 5%
  memory: 512Mi
```

Стало:

```
vm:
  cpu: 1
  coreFraction: 10%
  memory: 1Gi
```

Для бережного стенда не увеличивайте количество core без необходимости. Для демонстрации resize достаточно поднять coreFraction с 5% до 10% или RAM с 512Mi до 1Gi.

Пошаговое выполнение

```
kubectl get vm postgres-vm -n demo-prod -o yaml
git diff
git add gitops/environments/prod/values.yaml gitops/environments/prod/dvp-postgres-vm.yaml
git commit -m "Resize postgres VM"
git push
argocd app get demo-platform
kubectl get vm postgres-vm -n demo-prod
kubectl describe vm postgres-vm -n demo-prod
```

Что показывать в ArgoCD

- Diff по VM manifest.
- Sync operation.
- Возможное требование controlled restart, если платформа не меняет CPU/RAM live.

Что показывать в AWX

После изменения VM можно запустить AWX post-config: bootstrap, PostgreSQL tuning и validation. Это показывает, что ArgoCD меняет объект платформы, а AWX выполняет настройку внутри ОС.

Проверка через kubectl

```
kubectl get vm postgres-vm -n demo-prod
kubectl describe vm postgres-vm -n demo-prod
```

Ожидаемый результат

- VM manifest изменён через Git.
- ArgoCD видит и применяет изменение.
- Параметры VM соответствуют values.yaml или требуют controlled restart, если это ограничение платформы.

Rollback

```
git revert HEAD
git push
argocd app get demo-platform
```

Пояснение для демонстратора

На этом шаге важно не раздувать стенд. Для демонстрации достаточно минимальной VM и небольшого изменения coreFraction или RAM. Главная идея сохраняется: параметры инфраструктуры меняются через Git и проходят тот же audit trail.

Исторические сценарии: 05 AWX Post-Config

scenarios/05-awx-post-config.md

Цель

Показать, зачем нужен AWX вместе с ArgoCD.

Исходное состояние

- ArgoCD создал или изменил Kubernetes/DVP-ресурсы.
- VM postgres-vm доступна для Ansible по SSH.
- AWX inventory содержит postgres-vm.
- AWX token хранится в Kubernetes Secret, а не в Git.

Что меняем в Git

Добавляем или обновляем:

- gitops/awx/playbooks/bootstrap-vm.yml
- gitops/awx/playbooks/postgresql-tuning.yml
- gitops/awx/playbooks/validate-vm.yml
- gitops/awx/hooks/awx-postsync-job.yml
- gitops/awx/secrets/awx-token.example.yaml

Пошаговое выполнение

```
kubectl get vm postgres-vm -n demo-prod
kubectl get job -n demo-prod
kubectl logs job/awx-postsync-launch -n demo-prod
ansible-playbook gitops/awx/playbooks/validate-vm.yml
```

В AWX UI запустите Job Template вручную или покажите запуск через PostSync hook.

Что показывать в ArgoCD

- PostSync hook awx-postsync-launch.
- Sync wave 4.
- Что hook запускается после применения инфраструктуры.

Что показывать в AWX

- Inventory с postgres-vm.
- Job Template для bootstrap/postgresql tuning/validation.
- Stdout playbooks.
- Успешный финальный статус job.

Проверка через kubectl

```
kubectl get job awx-postsync-launch -n demo-prod
kubectl logs job/awx-postsync-launch -n demo-prod
```

Ожидаемый результат

- AWX Job завершился успешно.
- ОС внутри VM настроена.
- PostgreSQL параметры применены.
- Validation показал успешный статус.

Rollback

```
git revert HEAD
git push
argocd app get demo-platform
```

Если AWX job уже изменил ОС, выполните отдельный remediation playbook или повторите post-config с предыдущими значениями.

Пояснение для демонстратора

Здесь важно показать границу между декларативным и процедурным управлением. ArgoCD хорошо создаёт Kubernetes/DVP-объекты, но не должен превращаться в инструмент настройки ОС. AWX закрывает этот слой: пакеты, сервисы, PostgreSQL tuning и validation внутри гостевой системы.

Исторические сценарии: 06 Broken Release And Rollback

scenarios/06-broken-release-and-rollback.md

Цель

Показать безопасный rollback через Git.

Исходное состояние

- demo-app работает с образом nginx:1.27.
- ArgoCD Application demo-platform находится в Synced/Healthy.

Что меняем в Git

Файл:

```
gitops/environments/prod/values.yaml
```

Было:

```
app:
  image: nginx:1.27
```

Стало:

```
app:
  image: nginx:broken-demo-tag
```

Для текущего kustomize-примера также измените image в gitops/environments/prod/demo-app.yaml.

Пошаговое выполнение

```
git add gitops/environments/prod/values.yaml gitops/environments/prod/demo-app.yaml
git commit -m "Deploy broken image for rollback demo"
git push
argocd app get demo-platform
kubectl get pods -n demo-prod
kubectl describe pod -n demo-prod -l app=demo-app
git revert HEAD
git push
argocd app get demo-platform
kubectl get pods -n demo-prod
```

Что показывать в ArgoCD

- Degraded или Progressing состояние.
- Diff с несуществующим image tag.
- Возврат в Healthy после git revert.

Что показывать в AWX

AWX в этом сценарии не нужен. Это rollback приложения через GitOps.

Проверка через kubectl

```
kubectl get pods -n demo-prod
kubectl describe pod -n demo-prod -l app=demo-app
```

Ожидаемый результат

- ArgoCD показывает Degraded или Progressing.
- Kubernetes показывает ImagePullBackOff.
- После `git revert` приложение возвращается в Healthy.

Rollback

Основной rollback и есть часть сценария:

```
git revert HEAD
git push
```

Пояснение для демонстратора

Здесь полезно подчеркнуть, что ошибка не скрывается. Она видна в ArgoCD и Kubernetes events, а восстановление выполняется тем же процессом, что и доставка: через Git. История сохраняется, поэтому понятно, кто и когда изменил release.

Исторические сценарии: 07 Self-Service Tenant

scenarios/07-self-service-tenant.md

Цель

Показать self-service provisioning нового tenant через добавление каталога в Git.

Исходное состояние

- Platform team поддерживает tenant template.
- ArgoCD Application demo-platform синхронизирует prod-окружение.
- Новый внутренний заказчик называется customer-a.

Что меняем в Git

Добавляем каталог:

```
gitops/environments/prod/tenants/customer-a/
```

Файлы:

- namespace.yaml
- resourcequota.yaml
- limitrange.yaml
- rbac.yaml
- app.yaml
- vm.yaml
- README.md

Пошаговое выполнение

```
cp -r gitops/environments/prod/tenants/template gitops/environments/prod/tenants/customer-a
git add gitops/environments/prod/tenants/customer-a
git commit -m "Add customer-a tenant"
git push
argocd app get demo-platform
kubectl get ns customer-a
kubectl get resourcequota -n customer-a
kubectl get limitrange -n customer-a
kubectl get rolebinding -n customer-a
kubectl get deploy,svc,ingress -n customer-a
```

Если tenant уже добавлен в репозиторий, используйте существующий каталог как демонстрационный результат.

Что показывать в ArgoCD

- Новые ресурсы tenant в tree.
- Sync waves для namespace/RBAC/quota перед workload.
- Что platform team контролирует шаблон через Git.

Что показывать в AWX

При необходимости AWX может добавить tenant VM в inventory и выполнить bootstrap. В базовом сценарии достаточно показать, что post-config слой подключается тем же способом, что в сценарии 05.

Проверка через kubectl

```
kubectl get ns customer-a
kubectl get resourcequota -n customer-a
kubectl get limitrange -n customer-a
kubectl get rolebinding -n customer-a
kubectl get deploy,svc,ingress -n customer-a
```

Ожидаемый результат

- Namespace customer-a создан.
- Quota и LimitRange применены.
- RBAC настроен.
- Demo workload создан.
- Tenant подключён через GitOps.

Rollback

```
git revert HEAD
git push
argocd app get demo-platform
```

Если включен prune: true, ArgoCD удалит ресурсы tenant, которых больше нет в Git.

Пояснение для демонстратора

Этот сценарий хорошо работает для presale: новый tenant появляется не через набор ручных действий в кластере, а через стандартизированный каталог в Git. Platform team сохраняет контроль над шаблоном, а onboarding становится повторяемым и проверяемым.

Исторические сценарии: 08 Golden Image Management

scenarios/08-golden-image-management.md

Цель

Показать production-like сценарий управления golden image: администратор задаёт в Git ссылку на исходный cloud image, DVP импортирует его в кластер, затем builder VM подготавливается через AWX и из неё публикуется новая версия golden image.

Исходное состояние

- Argo CD Application demo-platform синхронизирует gitops/environments/prod.
- DVP CRD доступны в кластере.
- AWX Project Gitea demo геро смотрит на тот же Git-репозиторий.
- Для стенда используются минимальные ресурсы: 1 core, coreFraction: 5%, 512Mi RAM, disk 256Mi.

Что меняем в Git

Администратор указывает источник базового образа в Git:

```
gitops/environments/prod/golden-images/source-image.yaml
```

Ключевая часть:

```
spec:
  storage: ContainerRegistry
  dataSource:
    type: HTTP
    http:
      url: https://dl-cdn.alpinelinux.org/alpine/v3.23/releases/cloud/generic_alpine-3.23.3-x86_64-bios-cloudinit-r0.qcow2
```

Параметры также отражены в:

```
gitops/environments/prod/values.yaml
```

В текущем plain-YAML демо values.yaml является человекочитаемым паспортом параметров, а реальный DVP import выполняется из source-image.yaml.

Пошаговое выполнение

1. Показать source image в Git

Откройте Gitea:

```
http://gitea-awx.d8.kir.lab
```

Файл:

```
gitops/environments/prod/golden-images/source-image.yaml
```


Что сказать:

Администратор не скачивает image руками и не загружает его через UI. Он указывает URL исходного cloud image в Git. Argo CD применяет VirtualImage, а DVP импортирует образ в кластер.

2. Применить импорт через GitOps

Если файл уже в Git, достаточно дождаться Argo CD. Для ручного ускорения:

```
kubectl annotate application -n argocd demo-platform argocd.argoproj.io/refresh=hard --overwrite
```

Проверка:

```
kubectl get vi alpine-base-3-23-v1 -n demo-prod -o wide
```

Ожидаемо:

```
PHASE Ready  
PROGRESS 100%
```

3. Проверить builder disk и builder VM

```
kubectl get vd golden-builder-root -n demo-prod -o wide  
kubectl get vm golden-builder-vm -n demo-prod -o wide
```

Ожидаемо:

- golden-builder-root создан из alpine-base-3-23-v1;
- golden-builder-vm существует, но имеет runPolicy: Manual, чтобы не расходовать ресурсы без необходимости.

4. Запустить builder VM для customization

Для живого прогона можно временно изменить в Git:

```
gitops/environments/prod/golden-images/builder-vm.yaml
```

Было:

```
runPolicy: Manual
```

Стало:

```
runPolicy: AlwaysOnUnlessStoppedManually
```

Затем:

```
git add gitops/environments/prod/golden-images/builder-vm.yaml  
git commit -m "Start golden image builder VM"  
git push origin main  
git push dkp-gitea main
```

Проверка:

```
kubectl get vm golden-builder-vm -n demo-prod -o wide
```

5. Запустить AWX customization

В AWX создайте inventory host `golden-builder-vm` в группе:

```
golden_builder
```

Variables:

```
ansible_host: <IP golden-builder-vm>
```

Запустите playbook:

```
gitops/awx/playbooks/prepare-golden-image.yml
```

Он:

- устанавливает пакеты;
- включает `qemu-guest-agent`;
- пишет `demo config`;
- чистит кеш и логи;
- очищает `machine-id` там, где это применимо;
- удаляет SSH host keys для регенерации на первом boot.

6. Выполнить validation

Запустите:

```
gitops/awx/playbooks/validate-golden-image.yml
```

Проверяется:

- SSH/Python доступ;
- наличие `qemu-ga`;
- наличие `/etc/golden-image-demo.conf`.

7. Опубликовать новую версию golden image

После успешной подготовки и проверки примените manifest:

```
gitops/environments/prod/golden-images/publish-golden-image.example.yaml
```

В production лучше делать это отдельным Git commit после validation:

```
cp gitops/environments/prod/golden-images/publish-golden-image.example.yaml \
  gitops/environments/prod/golden-images/alpine-golden-3-23-v1.yaml
```

Добавьте новый файл в `golden-images/kustomization.yaml`, сделайте commit и push.

Проверка:

```
kubectl get vi alpine-golden-3-23-v1 -n demo-prod -o wide
```

8. Переключить потребителей

Дальше рабочие VM или VM templates переводятся с base image на новую версию golden image через Git:

```
objectRef:
  kind: VirtualImage
  name: alpine-golden-3-23-v1
```

Что показывать в Argo CD

- VirtualImage/alpine-base-3-23-v1 импортируется из URL.
- VirtualDisk/golden-builder-root создаётся из импортированного image.
- VirtualMachine/golden-builder-vm управляется GitOps.
- Новая версия golden image появляется как отдельный artifact после validation.

Что показывать в AWX

- Job prepare-golden-image.yml.
- Job validate-golden-image.yml.
- Stdout с установкой пакетов, включением qemu-guest-agent и validation summary.

Проверка через kubectl

```
kubectl get vi -n demo-prod -o wide
kubectl get vd golden-builder-root -n demo-prod -o wide
kubectl get vm golden-builder-vm -n demo-prod -o wide
kubectl describe vi alpine-base-3-23-v1 -n demo-prod
```

Ожидаемый результат

- DVP импортировал исходный image из URL, заданного в Git.
- Builder disk создан из imported image.
- Builder VM готова к customization через AWX.
- После validation можно опубликовать новую immutable-версию golden image.

Rollback

Rollback выполняется через Git:

```
git revert HEAD
git push origin main
git push dkp-gitea main
```

Для production-модели не перезаписывайте существующий golden image. Создавайте новую версию:

```
alpine-golden-3-23-v1
alpine-golden-3-23-v2
```

Тогда rollback означает переключение потребителей обратно на предыдущую версию.

Пояснение для демонстратора

На этом сценарии важно подчеркнуть, что golden image — это не ручной артефакт, загруженный через UI. Источник образа, builder VM, playbooks настройки и публикация версии описаны в Git. Argo CD отвечает за DVP-ресурсы, AWX — за изменение ОС внутри builder VM, а Git сохраняет историю: из какого URL был импортирован образ, каким playbook'ом он был подготовлен и на какую версию переключили потребители.

План рассказа для живого демо

0. Вводная

Что сказать:

В production одна из частых задач платформенной команды — управление golden image. Нужно регулярно брать новый исходный cloud image, добавлять в него корпоративные пакеты, агенты, конфиги, security baseline и публиковать новую версию для команд.

Ключевая мысль:

Мы хотим, чтобы выбор исходного image, процесс подготовки и переключение потребителей были управляемыми через Git, а не через ручную загрузку в UI.

1. Показать Git как точку выбора исходного образа

Открыть Gitea:

```
http://gitea-awx.d8.kir.lab
```

Открыть файл:

```
gitops/environments/prod/golden-images/source-image.yaml
```

Показать поле:

```
spec:
  dataSource:
    type: HTTP
    http:
      url: ...
```

Что сказать:

Вот ответ на вопрос “откуда берём новый изначальный образ”. Администратор меняет эту ссылку в Git. Например, сегодня это Alpine cloud image, завтра это новый Ubuntu cloud image или корпоративный base image, опубликованный в approved repository.

Ожидаемый вопрос аудитории:

А если URL поменяли?

Ответ:

Лучше не перезаписывать старый image с тем же именем, а создать новую версию: ubuntu-base-v1, ubuntu-base-v2. Тогда rollback — это обычный Git revert или переключение потребителей на предыдущую версию.

2. Показать Argo CD как механизм импорта в DVP

Открыть Argo CD:

```
http://argocd-awx.d8.kir.lab
```

Открыть Application:

```
demo-platform
```

Показать resource tree:

```
VirtualImage/alpine-base-3-23-v1
VirtualDisk/golden-builder-root
VirtualMachine/golden-builder-vm
```

Команды:

```
kubectl get vi alpine-base-3-23-v1 -n demo-prod -o wide
kubectl get vd golden-builder-root -n demo-prod -o wide
kubectl get vm golden-builder-vm -n demo-prod -o wide
```

Что сказать:

Argo CD не скачивает image сам. Он применяет DVP VirtualImage. После этого DVP-контроллер импортирует образ из URL и кладёт его во внутреннее хранилище.

3. Объяснить builder VM

Открыть файл:

```
gitops/environments/prod/golden-images/builder-vm.yaml
```

Показать:

```
runPolicy: Manual
cpu:
  cores: 1
  coreFraction: 5%
memory:
  size: 512Mi
```

Что сказать:

Builder VM — временная машина для подготовки образа. Она минимальная и по умолчанию в Manual, чтобы не расходовать ресурсы стенда. Когда нужно подготовить новую версию, мы включаем её через Git, запускаем AWX customization и после проверки публикуем image.

4. Показать AWX как слой customization

Открыть AWX:

```
http://awx-demo.d8.kir.lab
```

Открыть Project:

```
Gitea demo repo
```

Показать playbooks:

```
gitops/awx/playbooks/prepare-golden-image.yml
gitops/awx/playbooks/validate-golden-image.yml
```

Что сказать:

Argo CD создал builder VM, но не должен заниматься настройкой ОС. AWX выполняет то, что обычно входит в golden image baseline: пакеты, агенты, конфиги, очистка логов, подготовка к клонированию и validation.

5. Показать, что именно меняет AWX

Открыть в Gitea:

```
gitops/awx/playbooks/prepare-golden-image.yml
```

Показать задачи:

- установка qemu-guest-agent, chrony, curl, jq;
- включение qemu-guest-agent;
- запись /etc/golden-image-demo.conf;

- очистка cache/logs;
- очистка machine-id и SSH host keys.

Что сказать:

Это процедурная часть. Она версионруется в Git, но выполняется AWX внутри ОС builder VM.

6. Показать validation

Открыть:

```
gitops/awx/playbooks/validate-golden-image.yml
```

Что сказать:

Перед публикацией golden image мы проверяем, что VM доступна, qemu guest agent установлен, а demo config применён. В production сюда добавляются security checks, compliance checks, версии агентов и smoke tests.

7. Публикация golden image

Открыть:

```
gitops/environments/prod/golden-images/publish-golden-image.example.yaml
```

Что сказать:

Этот файл намеренно example. Мы не публикуем golden image автоматически до customization и validation. После успешной проверки создаётся отдельный Git commit, который добавляет новую версию image, например alpine-golden-3-23-v1.

Ключевая мысль:

Golden image должен быть immutable. Новая версия — новый объект. Не перетираем старый image.

8. Переключение потребителей

Показать идею:

```
objectRef:
  kind: VirtualImage
  name: alpine-golden-3-23-v1
```

Что сказать:

Рабочие VM или шаблоны VM переключаются на новую версию через Git. Если новая версия плохая, rollback — это Git revert на предыдущий image reference.

9. Финальная формулировка

В этом сценарии Git отвечает за выбор исходного image и версию golden image, Argo CD — за DVP-ресурсы и импорт, AWX — за настройку ОС внутри builder VM, а DVP — за хранение и запуск VM-образов. Это воспроизводимый pipeline управления golden image без ручной загрузки и без неявных изменений.

Исторические сценарии: 09 Self-Service Environment Request

scenarios/09-self-service-environment-request.md

Цель

Показать controlled self-service: разработчик создаёт запрос на временный стенд из заранее разрешённых профилей, а платформа через GitOps и AWX создаёт namespace, приложение, опциональную DVP VM, настройки и access info.

Архитектурная идея

Self-service UI или YAML request не создают ресурсы напрямую. Они создают Git artifact:

```
gitops/self-service/requests/<request-name>.yaml
```

После review/merge platform automation рендерит manifests в:

```
gitops/self-service/generated/<request-name>/
```

Argo CD применяет generated manifests, а AWX выполняет post-configuration для профилей с VM.

Исходное состояние

- Argo CD Application demo-platform синхронизирует gitops/environments/prod.
- gitops/self-service/catalog/ содержит approved profiles.
- self-service-ui/ содержит статическое web-приложение для генерации request YAML.
- Реальные секреты не хранятся в Git.

Что меняем в Git

Разработчик создаёт request:

```
gitops/self-service/requests/dev-alice-001.yaml
```

Пример:

```
apiVersion: demo.platform/v1
kind: EnvironmentRequest
metadata:
  name: dev-alice-001
spec:
  owner: alice
  team: payments
  profile: app-with-vm
  ttl: 8h
  access:
    exposeIngress: true
    sshToVm: false
  software:
    appImage: nginx:1.27
  vm:
    image: alpine-base-3-23-v1
    imageKind: ClusterVirtualImage
```

Профили

Approved catalog:

```
gitops/self-service/catalog/app-only.yaml
gitops/self-service/catalog/app-with-vm.yaml
gitops/self-service/catalog/app-with-postgres-vm.yaml
```

Что показать аудитории:

Разработчик не задаёт произвольные CPU/RAM/image. Он выбирает один из разрешённых профилей. Это controlled self-service, а не бесконтрольная выдача ресурсов.

Web UI

Открыть локально:

```
open self-service-ui/index.html
```

Что показать:

- выбор owner/team/environment name;
- выбор profile;
- TTL;
- image из allow-list;
- YAML EnvironmentRequest;
- Git commands для branch/commit/push.

Что сказать:

UI — это не второй control plane. Он помогает разработчику сформировать корректный GitOps-запрос. После этого всё равно остаются Git, review, merge, Argo CD и AWX.

Пошаговое выполнение

1. Показать каталог профилей

Откройте Gitea:

```
http://gitea-awx.d8.kir.lab
```

Файлы:

```
gitops/self-service/catalog/app-only.yaml
gitops/self-service/catalog/app-with-vm.yaml
gitops/self-service/catalog/app-with-postgres-vm.yaml
```

2. Сгенерировать request через UI

```
open self-service-ui/index.html
```

Выберите:

- owner: alice;
- team: payments;
- environment: dev-alice-001;

- profile: app-with-vm;
- TTL: 8h;
- app image: nginx:1.27;
- VM image: alpine-base-3-23-v1.
- VM image kind: ClusterVirtualImage, чтобы tenant namespace мог использовать утверждённый платформенный образ.
- VM в generated-примере запускается с runPolicy: AlwaysOn, но с минимальными ресурсами: 1 core, coreFraction: 5%, 512Mi.

Скопируйте YAML.

3. Создать Git request

```
git checkout -b request/dev-alice-001
mkdir -p gitops/self-service/requests
$EDITOR gitops/self-service/requests/dev-alice-001.yaml
git add gitops/self-service/requests/dev-alice-001.yaml
git commit -m "Request self-service environment dev-alice-001"
git push origin request/dev-alice-001
git push dkp-gitea request/dev-alice-001
```

В production дальше создаётся PR. В демо можно показать уже подготовленный generated пример:

```
gitops/self-service/generated/dev-alice-001/
```

4. Показать generated manifests

Файлы:

```
namespace.yaml
quota.yaml
rbac.yaml
app.yaml
vm.yaml
access-secret.example.yaml
ACCESS.md
```

Что сказать:

В production этот каталог должен генерироваться automation/CI/controller. В демо он добавлен заранее, чтобы показать результат обработки request.

5. Проверить Argo CD

```
kubectl get application -n argocd demo-platform
kubectl get ns dev-alice-001
kubectl get deploy,svc,ingress -n dev-alice-001
kubectl get vd,vm -n dev-alice-001 -o wide
```

6. Показать access info

Открыть:

```
gitops/self-service/generated/dev-alice-001/ACCESS.md
```

Что сказать:

Реальные креды не должны лежать в Git. Здесь только demo artifact и инструкция, где разработчик получает namespace, URL и способ доступа. В production это Vault, External Secrets, OIDC/RBAC или short-lived credentials.

7. AWX post-config

Для профиля `app-with-vm` AWX должен выполнить `post-config`:

- bootstrap VM;
- validation;
- при необходимости установку дополнительного ПО.

В текущем демо это можно показать как следующий автоматизируемый шаг через AWX API или Job Template.

Альтернативный путь: GitOps/YAML без web UI

Этот вариант показывает, что web portal не является обязательной точкой входа. Разработчик может создать такую же заявку обычным YAML-файлом в Git.

1. Получить репозиторий

```
git clone https://github.com/kirka1206/ArgoAWXk8sDVPdemo.git
cd ArgoAWXk8sDVPdemo
git remote add dkp-gitea http://gitea-awx.d8.kir.lab/codex/demo.git || true
```

Что сказать:

Для live-стенда важно пушить в Gitea, потому что Argo CD читает именно `http://gitea-awx.d8.kir.lab/codex/demo.git`.

2. Показать approved catalog

```
ls gitops/self-service/catalog
```

Ожидаемо:

```
app-only.yaml
app-with-vm.yaml
app-with-postgres-vm.yaml
```

Что сказать:

YAML-заявка не даёт разработчику произвольные ресурсы. Он всё равно выбирает `approved profile`.

3. Создать request YAML

```
cat > gitops/self-service/requests/dev-alice-yaml-demo.yaml <<'EOF'
apiVersion: demo.platform/v1
kind: EnvironmentRequest
metadata:
  name: dev-alice-yaml-demo
spec:
  owner: alice-koroleva
  email: alice.koroleva@demo.local
  groups:
    - payments-devs
  profile: app-with-vm
  purpose: demo
  ttl: 2h
  access:
    exposeIngress: true
    sshToVm: false
  software:
    appImage: nginx:1.27
  vm:
    image: alpine-base-3-23-v1
    imageKind: ClusterVirtualImage
EOF
```

Что объяснить:

- `metadata.name` станет именем заявки и будущего namespace;
- `owner`, `email`, `groups` фиксируют автора;
- `profile` выбирает approved template;
- `purpose` нужен для аудита и будущего cleanup/policy automation;
- `ttl` задаёт срок жизни;
- `appImage` и `vm.image` берутся из разрешённого набора.

4. Проверить diff и отправить в Git

```
git diff
git add gitops/self-service/requests/dev-alice-yaml-demo.yaml
git commit -m "Request self-service environment dev-alice-yaml-demo"
git push origin main
git push dkp-gitea main
```

Что сказать:

В production вместо прямого push в main должен быть branch/PR, policy validation и approve/merge. Для демо допустим прямой push, чтобы быстро показать цепочку.

5. Generated manifests

Production-like automation должна создать:

```
gitops/self-service/generated/dev-alice-yaml-demo/
```

В текущем стенде generated manifests создаёт portal/backend или демонстратор готовит их заранее.

Что сказать:

Request - это входной объект. Argo CD применяет конкретные Kubernetes/DVP manifests из generated/. Следующий архитектурный шаг проекта - controller/CI, который автоматически превращает request в generated manifests.

6. Проверить Argo CD и ресурсы

```
kubectl get application -n argocd demo-platform
NS=dev-alice-yaml-demo
kubectl get ns $NS
kubectl get deploy,svc,ingress -n $NS
kubectl get vd,vm -n $NS
```

Ожидаемый результат для app-with-vm:

- demo-platform в состоянии Synced/Healthy;
- namespace dev-alice-yaml-demo создан;
- demo-app доступен;
- ingress создан;
- DVP VirtualDisk и VirtualMachine созданы.

7. Rollback

```
git rm gitops/self-service/requests/dev-alice-yaml-demo.yaml
git rm -r gitops/self-service/generated/dev-alice-yaml-demo
git commit -m "Remove self-service environment dev-alice-yaml-demo"
git push dkp-gitea main
```

Что сказать:

Rollback и cleanup тоже проходят через Git. Это сохраняет историю и делает удаление таким же контролируемым, как создание.

Что показывать в Argo CD

- Новый namespace dev-alice-001.
- Quota/RBAC.
- App Deployment/Service/Ingress.
- DVP VM resources, если профиль включает VM.

Что показывать в AWX

- Что профиль определяет post-config playbooks.
- Что AWX не создаёт namespace/VM, а настраивает ОС после GitOps-создания.

Проверка через kubectl

```
kubectl get ns dev-alice-001
kubectl get resourcequota,limitrange,rolebinding -n dev-alice-001
kubectl get deploy,svc,ingress -n dev-alice-001
kubectl get vd,vm -n dev-alice-001 -o wide
```

Ожидаемый результат

- Request описан в Git.
- Generated manifests созданы из approved profile.
- Argo CD создал окружение.
- Разработчик получил namespace, endpoint и инструкцию по доступу.
- VM создаётся только для профилей, где она нужна.

Rollback / Cleanup

Удаление временного стенда должно быть Git-driven:

```
git rm -r gitops/self-service/generated/dev-alice-001
git commit -m "Remove self-service environment dev-alice-001"
git push origin main
git push dkr-gitea main
```

Для production добавляется TTL cleanup controller/job.

Пояснение для демонстратора

Self-service не означает, что разработчик получил cluster-admin. Он получил понятный интерфейс и каталог разрешённых профилей. Все изменения всё равно проходят через Git, а значит остаются review, история, rollback и политики. Web UI улучшает UX, но не заменяет GitOps.

Исторические сценарии: 10 Self-Service Portal

scenarios/10-self-service-portal.md

Цель

Показать, как разработчик создаёт тестовый стенд через web UI, но GitOps остаётся источником правды.

Исходное состояние

- Portal доступен по `https://selfservice-awx.d8.kir.lab`.
- Доступ закрыт через DKP DexAuthenticator.
- В Dex созданы демо-пользователи и группы.
- Backend имеет Kubernetes Secret с учёткой Gitea.
- Argo CD синхронизирует `gitops/environments/prod`.

Что делает пользователь

- Открывает `https://selfservice-awx.d8.kir.lab`.
- Логинится через Dex.
- Выбирает профиль стенда:
- `app-only`;
- `app-with-vm`;
- `app-with-postgres-vm`.
- Выбирает `purpose` и `TTL`.
- Нажимает Создать стенд.
- Ждёт статуса на странице.

Смысл

Профиль стенда — это не свободная форма заказа ресурсов, а заранее утверждённый шаблон.

- `app-only`: создаёт только namespace, RBAC, квоты, `demo-app`, `service` и `ingress`.
- `app-with-vm`: добавляет минимальную DVP VM из утверждённого `ClusterVirtualImage`.
- `app-with-postgres-vm`: добавляет VM как цель для PostgreSQL `post-configuration` через AWX.

Для PostgreSQL-профиля выберите версию 16, 17 или 18. После готовности VM в блоке результата скопируйте сформированную команду `d8 v ssh: portal` уже подставит пользователя `ansible`, имя VM, namespace и путь к локальному ключу.

- помогает аудиторам понять, зачем создан стенд;
- попадает в GitOps request и labels/annotations;
- может использоваться будущей `cleanup/policy automation`.

Что делает система

Portal не создаёт Kubernetes-ресурсы напрямую. Он создаёт Git artifacts в Gitea:

```
gitops/self-service/requests/<name>.yaml
gitops/self-service/generated/<name>/
```

После этого Argo CD применяет generated manifests.

Что показывать в Argo CD

- Application demo-platform.
- Новый namespace как managed resource.
- Generated Deployment, Service, Ingress.
- DVP VirtualDisk и VirtualMachine, если выбран VM-профиль.

Что показывать в AWX

Для VM-профилей объяснить, что после создания VM AWX выполняет post-configuration:

- bootstrap ОС;
- установка пакетов;
- настройка qemu-guest-agent;
- validation.

В текущей реализации portal создаёт инфраструктуру и VM, а AWX post-config остаётся отдельным демонстрационным шагом.

Проверка через kubectl

```
kubectl get deploy,svc,ingress -n self-service-portal
kubectl get dexauthenticator -n self-service-portal
kubectl get certificate -n self-service-portal self-service-portal
kubectl get application -n argocd demo-platform
kubectl get ns | grep dev-
kubectl get deploy,svc,ingress,vd,vm -n <generated-namespace>
```

Ожидаемый результат

- Пользователь видит только профили, разрешённые его группе.
- Имя стенда генерируется автоматически.
- В Gitea появляется request и generated manifests.
- Argo CD создаёт Kubernetes/DVP ресурсы.
- Portal показывает namespace, профиль, purpose, TTL, квоты, app параметры, service, ingress, VM/disk параметры и Git paths.

Rollback

Удалить generated каталог и request из Git, затем дождаться Argo CD prune:

```
git rm -r gitops/self-service/generated/<generated-namespace>
git rm gitops/self-service/requests/<generated-namespace>.yaml
git commit -m "Remove self-service environment <generated-namespace>"
git push dkp-gitea main
```

Пояснение для демонстратора

На этом шаге важно подчеркнуть, что UI не заменяет GitOps. Он только делает developer experience удобнее: пользователь выбирает разрешённые параметры, а система превращает это в Git commit. Вся

инфраструктура по-прежнему проходит через Git, Argo CD и аудируемую историю изменений.

Исторические сценарии: 11 Practicum End-to-End

scenarios/11-practicum-end-to-end.md

Цель

Показать три актуальных сценария нового стенда: выпуск golden image, заказ окружения через Git или Web и автоматическое удаление после TTL.

Исходное состояние

- context: practicum-tks-api.d8case.ru;
- namespace: practicum-tks;
- Argo CD Application practicum-demo — Synced/Healthy;
- active image: practicum-alpine-golden-3-23-v2;
- portal: https://selfservice-practicum.d8case.ru.

Golden image

- В Gitea откройте gitops/environments/practicum/golden-images/.
- Покажите URL source image, минимальный builder и versioned VirtualImage.
- В AWX откройте workflow Practicum Golden Image Build.
- Покажите успешный workflow job 40: prepare, validate, shutdown.
- В Argo CD покажите v1 и v2. Подчеркните, что v1 не изменялся.
- В catalog.yaml покажите отдельное переключение activeGoldenImage.

Проверка:

```
kubectl get vi,vd,vm -n practicum-tks -o wide
kubectl get cm practicum-golden-image-catalog -n practicum-tks -o yaml
```

Git self-service

Создайте JSON-formatted YAML в gitops/self-service/practicum/requests/<environment>.yaml:

```
{
  "apiVersion": "demo.practicum/v1",
  "kind": "EnvironmentRequest",
  "metadata": {"name": "practicum-env-marina-demo-001"},
  "spec": {
    "owner": "marina-volkova-practicum",
    "email": "marina.volkova.practicum@demo.local",
    "groups": ["practicum-qa-devs"],
    "profile": "app-with-vm",
    "purpose": "demo",
    "ttl": "2h",
    "createdAt": "2026-06-09T12:00:00Z"
  }
}
```

После push controller создаёт generated-каталог, Argo CD применяет app/VM, AWX запускает post-config. Все ресурсы остаются в practicum-tks.

Web self-service

- Откройте <https://selfservice-practicum.d8case.ru>.
- Войдите пользователем Alice, Boris или Marina.
- Выберите разрешённый профиль, purpose и TTL.
- Нажмите Отправить заявку.
- Покажите environment ID отдельно от namespace.
- Дождитесь Ready и успешного AWX job.

Проверенный результат: practicum-env-alice-feature-019354, приложение 1/1, VM Running, IP 10.66.0.46, AgentReady=True, AWX job 55 successful.

TTL cleanup

Controller переводит request в Expired, удаляет environment из generated kustomization, архивирует request и записывает Cleaned. Argo CD prune удаляет только объекты с label данного environment.

```
kubectl get all,vd,vm -n practicum-tks \
  -l demo.practicum/environment=<environment-id>
```

Namespace, Argo CD, AWX, Gitea, portal и другие окружения сохраняются.

Пояснение для демонстратора

Разработчик не получает право создавать VM или менять Kubernetes напрямую. Он формулирует намерение через утверждённый request. Git хранит намерение и результат генерации, Argo CD приводит кластер к desired state, DVP создаёт VM, а AWX отвечает за процедурную настройку гостевой ОС. TTL реализован тем же путём: сначала меняется Git, затем Argo CD безопасно удаляет только принадлежащие заявке ресурсы.

Исторические сценарии: 12 DVP VM Drift Correction

scenarios/12-dvp-vm-drift-correction.md

Цель

Показать, как Argo CD обнаруживает ручное изменение DVP VirtualMachine и возвращает её спецификацию к состоянию из Git.

Сценарий одновременно объясняет границу ответственности:

- Git и Argo CD управляют декларативной конфигурацией VM;
- DVP выполняет операции жизненного цикла VM;
- AWX настраивает гостевую ОС;
- RBAC ограничивает опасные ручные операции.

Исходное состояние

- Application practicum-demo находится в состоянии Synced/Healthy.
- В namespace practicum-tks есть активная self-service VM.
- VM и её VirtualDisk имеют label demo.practicum/environment.
- В Application включено:

```
syncPolicy:
  automated:
    prune: true
    selfHeal: true
```

Выберите environment с VM:

```
kubectl get vm -n practicum-tks \
  -l demo.practicum/environment \
  -o custom-columns='VM:.metadata.name,OWNER:.metadata.labels.demo\practicum/owner,CPU:.spec.cpu.cores,FRACTION:.spec.cpu.coreFraction,MEMORY:.spec.memory.size,PHASE:.status.phase'
```

Для команд ниже задайте имя:

```
export VM_NAME=<имя-vm>
export APP_NAME=practicum-demo
export NAMESPACE=practicum-tks
```

Важное различие: operation и drift

Команды:

```
d8 v stop "$VM_NAME" -n "$NAMESPACE"
d8 v start "$VM_NAME" -n "$NAMESPACE"
d8 v restart "$VM_NAME" -n "$NAMESPACE"
```

создают DVP VirtualMachineOperation. Они не обязаны изменять .spec VirtualMachine, поэтому Argo CD может остаться Synced.

При runPolicy: AlwaysOnUnlessStoppedManually ручная остановка является разрешённым состоянием DVP. Для гарантированной демонстрации drift меняйте поле, которое хранится в Git: CPU или

RAM.

Что меняем в Git

В Git ничего не меняем. Это намеренное ручное изменение live-ресурса в обход GitOps.

Сначала сохраните ожидаемые значения:

```
kubectl get vm "$VM_NAME" -n "$NAMESPACE" \
-o jsonpath='{.spec.cpu.cores} coreFraction={.spec.cpu.coreFraction}
memory={.spec.memory.size}{"\n"}'
```

Пошаговое выполнение

1. Открыть интерфейсы

Откройте:

- Argo CD: <http://argocd-practicum.d8case.ru>;
- DKP: проект `practicum-tks` → Виртуализация → Виртуальные машины.

В Argo CD откройте Application `practicum-demo` и найдите выбранную VM в дереве ресурсов.

2. Запустить наблюдение

В первом терминале:

```
kubectl get application "$APP_NAME" -n "$NAMESPACE" -w
```

Во втором терминале:

```
kubectl get vm "$VM_NAME" -n "$NAMESPACE" -w
```

3. Создать безопасный drift

Измените только RAM:

```
kubectl patch vm "$VM_NAME" -n "$NAMESPACE" \
--type merge \
-p '{"spec":{"memory":{"size":"640Mi"}}}'
```

Альтернативно можно изменить `coreFraction`:

```
kubectl patch vm "$VM_NAME" -n "$NAMESPACE" \
--type merge \
-p '{"spec":{"cpu":{"coreFraction":"10%"}}}'
```

Не меняйте оба параметра одновременно: аудитории проще увидеть один diff.

4. Показать временное расхождение

```
kubectl get vm "$VM_NAME" -n "$NAMESPACE" \
-o jsonpath='{coreFraction={.spec.cpu.coreFraction} memory={.spec.memory.size}
needRestart={.status.restartAwaitingChanges}{"\n"}'
```

В Argo CD покажите:

- ресурс VM в состоянии OutOfSync;
- diff между live-состоянием и Git;
- автоматическую операцию self-heal.

Состояние OutOfSync может быть коротким: self-heal обычно запускается через несколько секунд.

5. Дождаться восстановления

```
until [[ "$(kubectl get application "$APP_NAME" -n "$NAMESPACE" \
-o jsonpath='{.status.sync.status}')" == "Synced" ]]; do
  kubectl get application "$APP_NAME" -n "$NAMESPACE"
  sleep 2
done
```

Проверьте параметры:

```
kubectl get vm "$VM_NAME" -n "$NAMESPACE" \
-o jsonpath='{.cores={.spec.cpu.cores} coreFraction={.spec.cpu.coreFraction}
memory={.spec.memory.size} phase={.status.phase}}'\n'
```

Argo CD должен вернуть значения из generated-манифеста Git.

6. Проверить необходимость перезапуска

Некоторые изменения ресурсов VM применяются только после controlled restart. Проверьте:

```
kubectl get vm "$VM_NAME" -n "$NAMESPACE" -o yaml |
sed -n '/restartAwaitingChanges/,+10p'
```

Если DVP показывает ожидающие применения изменения, выполните штатную операцию:

```
d8 v restart "$VM_NAME" -n "$NAMESPACE"
```

Restart является DVP-операцией и сам по себе не должен использоваться как пример configuration drift.

Что показывать в Argo CD

- Application до изменения: Synced/Healthy.
- Diff по VirtualMachine.spec.memory.size или VirtualMachine.spec.cpu.coreFraction.
- Кратковременный OutOfSync.
- Автоматический self-heal без изменения Git.
- Возврат в Synced/Healthy.

Если OutOfSync не успели увидеть в общем списке, откройте историю операций, events Application и diff конкретного ресурса VM.

Что показывать в DVP

- имя VM и владельца из labels;
- исходные CPU/RAM;
- изменённое live-значение;
- состояние Need Restart, если оно появилось;
- итоговые параметры после self-heal;
- отдельный объект VirtualMachineOperation при controlled restart.

Что показывать в AWX

В основном сценарии AWX не запускается: изменение CPU/RAM относится к декларативной конфигурации DVP, а не к настройке гостевой ОС.

Это важная граница:

- Argo CD восстанавливает VM-манифест;
- DVP применяет инфраструктурное состояние;
- AWX нужен, когда после создания или изменения VM требуется процедура внутри ОС.

Текущий controller не гарантирует повторный запуск уже успешного AWX job после ручного удаления и пересоздания VM. Для production-процесса нужен отдельный reconfigure trigger, основанный на новом VM UID, generation или Git revision.

Проверка через kubectl

```
kubectl get application "$APP_NAME" -n "$NAMESPACE"

kubectl get vm "$VM_NAME" -n "$NAMESPACE" \
  -o custom-columns='NAME:.metadata.name,PHASE:.status.phase,CPU:.spec.cpu.cores,FRACTION:.spec.\
  cpu.coreFraction,MEMORY:.spec.memory.size,IP:.status.ipAddress'

kubectl get events -n "$NAMESPACE" \
  --field-selector involvedObject.name="$VM_NAME" \
  --sort-by=.lastTimestamp
```

Опционально: удаление только VM

Этот блок показывайте только на временном environment, который можно пересоздать.

Сначала зафиксируйте диск:

```
export DISK_NAME="$(kubectl get vm "$VM_NAME" -n "$NAMESPACE" \
  -o jsonpath='{.spec.blockDeviceRefs[0].name}')"

kubectl get vd "$DISK_NAME" -n "$NAMESPACE" -o wide
```

Удалите только VirtualMachine:

```
kubectl delete vm "$VM_NAME" -n "$NAMESPACE"
```

Не удаляйте VirtualDisk.

Ожидаемая цепочка:

```
VirtualMachine удалена
→ Argo CD обнаруживает отсутствие tracked-ресурса
→ Application становится OutOfSync
→ selfHeal создаёт VirtualMachine заново
→ существующий VirtualDisk подключается повторно
→ DVP запускает VM
→ Application возвращается в Synced/Healthy
```

Проверка:

```
kubectl get vm,vd -n "$NAMESPACE" \
  -l demo.practicum/environment
```

Сохранение содержимого ОС зависит от сохранности и повторного подключения того же VirtualDisk.

Что нельзя делать в демонстрации

Не выполняйте:

```
kubectl delete vd "$DISK_NAME" -n "$NAMESPACE"
```

Удаление provisioned VirtualDisk может необратимо удалить данные. Argo CD создаст новый объект диска из golden image, но это будет чистая пересборка, а не восстановление пользовательских данных.

Также нельзя циклически удалять ресурсы быстрее, чем Argo CD завершает self-heal: это создаёт гонку контроллеров и делает результат демонстрации непредсказуемым.

Ожидаемый результат

- ручное изменение CPU/RAM обнаружено;
- Git не изменялся;
- Argo CD восстановил спецификацию VM;
- Application вернулся в Synced/Healthy;
- данные на диске не затронуты;
- при необходимости выполнен controlled restart через DVP operation.

Rollback

Для изменения CPU/RAM отдельный rollback не нужен: self-heal уже возвращает desired state из Git.

Если self-heal отключён:

```
argocd app sync "$APP_NAME"
```

Если VM была удалена и не пересоздалась:

```
kubectl annotate application "$APP_NAME" -n "$NAMESPACE" \
  argocd.argoproj.io/refresh=hard --overwrite
argocd app sync "$APP_NAME"
```

Не создавайте VM вручную другим манифестом: дождитесь применения того же desired state из Git.

Рекомендации по RBAC

Для нормальной эксплуатации:

- разрешить Victor start/stop/restart, console, VNC и snapshots;
- запретить удаление GitOps-managed VirtualDisk;
- удаление GitOps-managed VM разрешать только в контролируемом break-glass процессе;
- отделять ручные VM от GitOps-managed VM namespace или labels;
- запретить Victor менять VirtualImage, ClusterVirtualImage, Secrets, Deployments и RBAC;
- применять admission policy по label demo.practicum/environment, если одного RBAC недостаточно.

RBAC Kubernetes не умеет ограничивать delete по label. Для различения GitOps-managed и ручных VM надёжнее использовать отдельные namespaces либо admission policy.

Пояснение для демонстратора

На этом шаге важно подчеркнуть, что Argo CD контролирует не сам процесс внутри виртуальной машины, а декларативный объект DVP в Kubernetes API. Victor может выполнять разрешённые операции жизненного цикла, но ручное изменение CPU или RAM нарушает desired state из Git. Argo CD обнаруживает это расхождение и возвращает VM к утверждённой конфигурации.

Self-heal снижает риск случайного drift, но не заменяет RBAC и защиту данных. Особенно это важно для дисков: VM можно пересоздать, а удалённые данные автоматически вернуть нельзя.

Исторические сценарии: 13 Manual Environment Lifecycle

scenarios/13-manual-environment-lifecycle.md

Цель

Показать досрочное удаление и операции с DVP VM без нарушения GitOps: портал создаёт декларативное действие в Git, controller изменяет desired state, а Argo CD применяет изменение или выполняет prune.

Исходное состояние

- Application practicum-demo имеет Synced/Healthy;
- существует временный tenant environment с VM;
- пользователь входит через selfservice-practicum.d8case.ru;
- Victor входит через vm-admin-practicum.d8case.ru;
- golden builder VM не являются tenant environments и не отображаются Victor.

Что меняем в Git

Портал создаёт объект:

```
apiVersion: demo.practicum/v1
kind: EnvironmentAction
metadata:
  name: <environment>-delete-vm-<suffix>
spec:
  environment: <environment>
  action: delete-vm
  actor: <user-or-victor>
  actorEmail: <email>
  reason: <reason>
  requestedAt: <UTC timestamp>
```

Поддерживаются delete-vm, delete-environment, start-vm, stop-vm и restart-vm.

Пошаговое выполнение: пользователь

- Откройте <https://selfservice-practicum.d8case.ru>.
- Перейдите в Мои стенды.
- Убедитесь, что чужие environments не отображаются.
- Нажмите Удалить VM у собственного VM-профиля.
- В первом диалоге проверьте Environment ID и предупреждение.
- Подтвердите операцию второй кнопкой.
- Наблюдайте VMDeleting.
- После Argo CD prune убедитесь, что VM и VirtualDisk исчезли, а приложение осталось 1/1.
- Нажмите Удалить стенд и подтвердите.
- Дождитесь Deleting -> Cleaned.

Пошаговое выполнение: Victor

- Откройте `https://vm-admin-practicum.d8case.ru`.
- Войдите как `victor.melnikov.practicum@demo.local`.
- Покажите KPI, фильтры и список `tenant environments`.
- Убедитесь, что `golden builder VM` отсутствуют.
- Выберите `tenant VM` и выполните `Stop`, указав обязательную причину.
- Покажите `EnvironmentAction` и `VirtualMachineOperation` в дереве Argo CD.
- После завершения выполните `Start`, затем `Restart`.
- Для тестового стенда выполните `Удалить` стенд.
- Проверьте `audit: actor, reason, Git commit` и `outcome`.

Что показывать в Argo CD

- Application временно становится `OutOfSync/Progressing`;
- появляется `versioned VirtualMachineOperation` для `start/stop/restart`;
- после завершения operation удаляется из `desired state`;
- `delete VM/environment` выполняется через `prune`;
- итоговое состояние — `Synced/Healthy`.

Что показывать в AWX

- номер последнего `post-config job`;
- при удалении во время активного `job controller` запрашивает `cancel`;
- `lifecycle actions` не запускают повторный `post-config` без новой VM.

Проверка через kubectl

```
kubectl get application -n practicum-tks practicum-demo
kubectl get deploy,svc,ingress,vd,vm,vmap -n practicum-tks \
  -l demo.practicum/environment=<environment-id>
```

Ожидаемый результат

- пользователь управляет только собственными стендами;
- Victor управляет `tenant environments` всех пользователей;
- VM и диск удаляются вместе, приложение может быть сохранено;
- весь стенд удаляется только после изменения Git;
- namespace `practicum-tks`, `golden images` и `platform workloads` сохраняются;
- Git содержит полный `audit` действий.

Rollback

Удалённый `VirtualDisk` восстановить нельзя. Для возврата VM создайте новый `EnvironmentRequest` из активного `golden image`. Остановленную VM можно запустить новым `start-vm` action.

Пояснение для демонстратора

На этом шаге важно подчеркнуть, что портал не получил обходной путь к Kubernetes. Даже административное действие Victor сначала становится декларативным объектом в Git. Благодаря этому

Argo CD не борется с оператором, а исполняет новое desired state, а история пользователя, причины и результата остаётся воспроизводимой и проверяемой.

Операционный runbook

docs/operations.ru.md

Контуры стенда

В проекте есть два рабочих контура:

- `ansible-os-pods`: локальный pod-only сценарий для Docker Desktop и базовой демонстрации Argo CD + AWX.
- `demo-platform`: расширенный DKP/DVP сценарий с `demo-app`, `tenant customer-a` и минимальной DVP VM `postgres-vm`.

Для переноса на другой DKP/DVP стенд используйте:

- Пререквизиты стенда ([prerequisites.ru.md](#));
- Детальный план переноса ([migration-plan.ru.md](#)).

Запуск локального стенда

```
./scripts/bootstrap.sh
```

Скрипт устанавливает:

- Gitea;
- Argo CD;
- AWX;
- demo Linux pod'ы;
- AWX inventory, project, credentials, execution environment и job template.

Запуск DKP/DVP профиля

```
./scripts/deploy-dkp.sh
```

Проверить kube-context:

```
kubectl config current-context
```

Ожидаемый context:

```
codex-api.d8.kir.lab
```

Для нового стенда передайте параметры:

```
CONTEXT=<target-context> \  
GITEA_HOST=gitea-awx.<target-domain> \  
ARGOCD_HOST=argocd-awx.<target-domain> \  
AWX_HOST=awx-demo.<target-domain> \  
./scripts/deploy-dkp.sh
```

Проверить Ingress:

```
kubectl get ingress -A | grep -E 'gitea-awx|argocd-awx|awx-demo'
```

Повторно открыть локальные интерфейсы

```
./scripts/port-forward.sh
```

Локальные URL:

- Gitea: <http://localhost:3001>
- Argo CD: <http://localhost:3000>
- AWX: <http://localhost:3002>

DKP URL:

- Gitea: <http://gitea-awx.d8.kir.lab>
- Argo CD: <http://argocd-awx.d8.kir.lab>
- AWX: <http://awx-demo.d8.kir.lab>

Запуск AWX job

Локально:

```
./scripts/run-demo-job.sh
```

В DKP:

```
AWX_URL=http://awx-demo.d8.kir.lab ./scripts/run-demo-job.sh
```

Скрипт:

- находит AWX job template Configure OS pods;
- запускает job;
- ждёт финальный статус;
- выводит stdout;
- проверяет marker-файлы в pod'ax demo-os.

Проверка Argo CD

```
kubectl get application -n argocd
kubectl get application -n argocd ansible-os-pods
kubectl get application -n argocd demo-platform
```

Ожидаемо:

ansible-os-pods	Synced	Healthy
demo-platform	Synced	Healthy

Проверка pod-only сценария

```
kubectl get pods,svc -n demo-os
kubectl exec -n demo-os deploy/ol-node-1 -- cat /etc/ansible-managed-by-awx
kubectl exec -n demo-os deploy/ol-node-2 -- cat /etc/ansible-managed-by-awx
```

Проверка DVP-сценария

```
kubectl get deploy,svc,ingress -n demo-prod
kubectl get vi,vd,vm -n demo-prod -o wide
kubectl describe vm postgres-vm -n demo-prod
```

Ожидаемые параметры VM:

```
postgres-vm Running
1 core
coreFraction 5%
memory 512Mi
disk 256Mi
```

Проверка tenant

```
kubectl get ns customer-a
kubectl get resourcequota -n customer-a
kubectl get limitrange -n customer-a
kubectl get rolebinding -n customer-a
kubectl get deploy,svc,ingress -n customer-a
```

Частые проблемы

401 при запуске AWX job

Скорее всего, скрипт попал в старый AWX через `localhost:3002`.

Проверить:

```
curl -fsS http://localhost:3002/api/v2/ping/
curl -fsS http://awx-demo.d8.kir.lab/api/v2/ping/
```

Для DKP используйте:

```
AWX_URL=http://awx-demo.d8.kir.lab ./scripts/run-demo-job.sh
```

DVP VM зависла в Pending

Проверить image, disk, VM и events:

```
kubectl get vi,vd,vm -n demo-prod -o wide
kubectl describe vi demo-alpine-cloud -n demo-prod
kubectl describe vd postgres-vm-root -n demo-prod
kubectl describe vm postgres-vm -n demo-prod
kubectl get events -n demo-prod --sort-by=.lastTimestamp
```

На первом запуске DVP скачивает image и импортирует disk. Это может занять несколько минут.

AWX долго стартует

```
kubectl get job,pods -n awx
kubectl logs -n awx job/awx-demo-migration-24.6.1 --tail=100
```

Argo CD Application не Synced

```
kubectl describe application -n argocd demo-platform
kubectl describe application -n argocd ansible-os-pods
```

Типовые причины:

- Gitea репо не обновлён;
- указан неверный path;
- CRD DVP ещё не готов;
- namespace или StorageClass недоступны.
- gitops/self-service/generated/kustomization.yaml содержит невалидную комбинацию resources: [] и вложенных – <name>.

Валидный пустой вариант:

```
resources: []
```

Валидный вариант со стендами:

```
resources:
  - dev-example
```

Master node scheduling

В стендовом кластере taint с master node снят, чтобы scheduler мог использовать ресурсы dmaster.

Проверка:

```
kubectl get nodes -o json | jq -r '.items[] | .metadata.name as $n | "\($n)\t\(.spec.taints // [])"'
kubectl get pods -A -o wide | grep dmaster
```

Rollback

Для сценарных изменений:

```
git revert HEAD
git push
argocd app get demo-platform
```

Очистка

Локальный cleanup:

```
./scripts/destroy.sh
```

Перед удалением DVP-ресурсов проверьте, что они не нужны:

```
kubectl get vi,vd,vm -n demo-prod
```

Use cases

docs/use-cases.ru.md

Документ описывает, что именно показывать аудитории на стенде Argo CD + AWX + Kubernetes + DVP.

1. GitOps-доставка Kubernetes-ресурсов

Инженер меняет манифесты в Git. Argo CD обнаруживает изменение и приводит кластер к описанному состоянию.

В локальном сценарии это:

- namespace demo-os;
- pod'ы ol-node-1, ol-node-2;
- Services для SSH.

В DKP/DVP сценарии это:

- namespace demo-prod;
- Deployment/Service/Ingress demo-app;
- RBAC, quotas, limit ranges;
- tenant customer-a.

Ценность: состояние инфраструктуры версионизируется, проходит review и может быть восстановлено из Git.

2. GitOps-доставка DVP VM

Argo CD синхронизирует реальные DVP CRD:

- VirtualImage demo-alpine-cloud;
- VirtualDisk postgres-vm-root;
- VirtualMachine postgres-vm.

VM создана с минимальными ресурсами:

```
1 core
coreFraction 5%
memory 512Mi
disk 256Mi
```

Ценность: виртуальная машина становится частью того же GitOps-контура, что и Kubernetes workload'ы.

3. Настройка ОС после создания ресурса

После того как Argo CD создал workload или VM, AWX запускает Ansible playbook и выполняет настройку внутри ОС.

В pod-only сценарии playbook:

- собирает facts;
- записывает /etc/ansible-managed-by-awx;
- устанавливает htop;
- выводит результат.

В DVP/VM сценарии подготовлены playbooks:

- `gitops/awx/playbooks/bootstrap-vm.yml`;
- `gitops/awx/playbooks/postgresql-tuning.yml`;
- `gitops/awx/playbooks/validate-vm.yml`.

Ценность: lifecycle Kubernetes/DVP-объектов и lifecycle ОС разделены, но связаны в единую операционную модель.

4. Configuration drift и self-healing

Администратор вручную меняет состояние в кластере, например:

```
kubectl scale deployment demo-app --replicas=10 -n demo-prod
```

Argo CD видит расхождение с Git и возвращает Deployment к значению из репозитория.

Ценность: Git остаётся источником истины, даже если кто-то изменил кластер вручную.

5. Rollback через Git

В Git попадает ошибка, например несуществующий image tag:

```
app:
  image: nginx:broken-demo-tag
```

Kubernetes показывает ImagePullBackOff, Argo CD показывает Progressing или Degraded, затем команда делает:

```
git revert HEAD
git push
```

Ценность: восстановление выполняется тем же процессом, что и доставка, а история изменений сохраняется.

6. Self-service tenant onboarding

Platform team добавляет каталог:

```
gitops/environments/prod/tenants/customer-a/
```

Argo CD создаёт:

- namespace;
- ResourceQuota;
- LimitRange;
- RBAC;
- starter workload;
- опциональный VM template.

Ценность: onboarding нового внутреннего заказчика становится повторяемым и стандартизированным.

7. Единый audit trail

Gitea хранит:

- Kubernetes manifests;

- DVP manifests;
- Ansible playbooks;
- сценарии демонстрации;
- документацию.

Argo CD показывает sync status и drift. AWX показывает историю job'ов, stdout и результат по host'ам.

Ценность: можно объяснить, что изменилось, каким инструментом и на каком уровне.

8. Управление golden image

Администратор указывает в Git URL исходного cloud image:

```
gitops/environments/prod/golden-images/source-image.yaml
```

Argo CD применяет VirtualImage, DVP импортирует образ в кластер, затем создаётся builder disk и builder VM. AWX выполняет customization и validation, после чего публикуется новая версия golden image.

Ценность: golden image становится воспроизводимым артефактом с историей изменений, а не ручной загрузкой через UI.

9. Self-service создание временного стенда

Разработчик выбирает профиль стенда из approved catalog и создаёт Git request. Это можно сделать YAML-файлом или через web UI:

```
open self-service-ui/index.html
```

Профили:

- app-only;
- app-with-vm;
- app-with-postgres-vm.

После merge или отправки заявки через portal backend generated manifests появляются в Git. Argo CD применяет их в Kubernetes/DVP, а AWX выполняет post-configuration для профилей с VM.

Ценность: разработчик получает быстрый путь к стенду, а platform team сохраняет контроль через catalog, Git review, quotas, TTL и allow-list образов.

Демонстрационный сценарий

Подготовка

Проверить контекст:

```
kubectrl config current-context
```

Для DKP ожидается:

```
codex-api.d8.kir.lab
```

Проверить приложения:

```
kubectrl get application -n argocd
```

Ожидаемо:

ansible-os-pods	Synced	Healthy
demo-platform	Synced	Healthy

Шаг 1. Показать Git как source of truth

Открыть Gitea:

```
http://gitea-awx.d8.kir.lab
```

Показать файлы:

- gitops/demo-manifests/os-nodes.yaml;
- gitops/environments/prod/dvp-postgres-vm.yaml;
- gitops/environments/prod/values.yaml;
- gitops/awx/playbooks/validate-vm.yml;
- scenarios/.

Сообщение для аудитории: в одном Git-репозитории лежит декларативное состояние платформы, сценарии изменений и Ansible-автоматизация.

Шаг 2. Показать Argo CD

Открыть:

```
http://argocd-awx.d8.kir.lab
```

Показать Applications:

- ansible-os-pods;
- demo-platform.

Проверить из терминала:

```
kubectl get deploy,svc,ingress -n demo-prod
kubectl get vi,vd,vm -n demo-prod -o wide
```

Сообщение для аудитории: Argo CD синхронизирует и обычные Kubernetes-объекты, и DVP CRD.

Шаг 3. Показать DVP VM

```
kubectl describe vm postgres-vm -n demo-prod
```

Что подчеркнуть:

- VM создана из Git;
- ресурсы минимальны для стенда;
- disk и image тоже управляются как Kubernetes CRD;
- VM находится в Running.

Шаг 4. Показать AWX

Открыть:

```
http://awx-demo.d8.kir.lab
```

Запустить pod-only job:

```
AWX_URL=http://awx-demo.d8.kir.lab ./scripts/run-demo-job.sh
```

Показать:

- Gathering Facts;
- marker /etc/ansible-managed-by-awx;
- recap failed=0.

Сообщение для аудитории: AWX не заменяет Argo CD. Он работает на другом уровне: внутри ОС.

Шаг 5. Показать drift correction

```
kubectl scale deployment demo-app --replicas=10 -n demo-prod  
argocd app get demo-platform  
kubectl get deploy demo-app -n demo-prod
```

После self-heal replicas должны вернуться к значению из Git.

Шаг 6. Показать rollback

Следовать сценарию:

```
scenarios/06-broken-release-and-rollback.md
```

Главная мысль: ошибка и восстановление проходят через Git, а не через ручное исправление в кластере.

Шаг 7. Показать tenant onboarding

```
kubectl get ns customer-a  
kubectl get resourcequota,limitrange,rolebinding -n customer-a  
kubectl get deploy,svc,ingress -n customer-a
```

Сообщение для аудитории: новый tenant появляется через каталог в Git, а не через ручной набор действий.

Финальная формулировка

Argo CD отвечает за долгоживущее декларативное состояние Kubernetes/DVP-платформы. AWX отвечает за процедурную настройку внутри ОС. Вместе они дают воспроизводимый путь от Git commit до работающего и проверенного workload'a.

Self-service GitOps

docs/self-service.ru.md

Self-service сценарий показывает, как разработчик может запросить временный стенд из approved catalog без ручного обращения к администратору.

Принцип

Web UI и YAML request не создают ресурсы напрямую. Они создают Git artifact:

```
gitops/self-service/requests/<request-name>.yaml
```

После review/merge automation или portal backend генерирует manifests, Argo CD применяет их, AWX выполняет post-configuration для VM-профилей.

Почему это безопаснее прямого UI в кластер

- разработчик выбирает только разрешённый profile;
- CPU/RAM/image/TTL ограничены catalog'ом;
- DVP VM создаётся из утверждённого ClusterVirtualImage, доступного tenant namespace;
- все изменения проходят через Git;
- rollback и cleanup делаются через Git;
- реальные credentials не хранятся в Git.

Web UI

Открыть:

```
open self-service-ui/index.html
```

UI генерирует:

- EnvironmentRequest YAML;
- команды для branch/commit/push;
- список ресурсов, которые будут созданы после merge.

GitOps/YAML путь

Этот вариант нужен, чтобы показать чистую GitOps-модель без web portal.

Разработчик создаёт YAML-заявку:

```
gitops/self-service/requests/<request-name>.yaml
```

Пример:

```

apiVersion: demo.platform/v1
kind: EnvironmentRequest
metadata:
  name: dev-alice-yaml-demo
spec:
  owner: alice-koroleva
  email: alice.koroleva@demo.local
  groups:
    - payments-devs
  profile: app-with-vm
  purpose: demo
  ttl: 2h
  access:
    exposeIngress: true
    sshToVm: false
  software:
    appImage: nginx:1.27
  vm:
    image: alpine-base-3-23-v1
    imageKind: ClusterVirtualImage

```

После commit/push automation должна сгенерировать:

```
gitops/self-service/generated/<request-name>/
```

В текущем live-демо portal backend пишет сразу в Gitea main: создаёт request, generated manifests и обновляет gitops/self-service/generated/kustomization.yaml. Production-like следующий шаг - branch/PR flow или отдельный controller/CI, который валидирует EnvironmentRequest и генерирует manifests автоматически.

Важно: если generated-стендов нет, top-level файл должен быть валидным пустым Kustomization:

```

apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources: []

```

Если стенды есть, используйте многострочный список:

```

resources:
  - dev-alice-koroleva-demo-c3aa

```

Проверка:

```

kubectl get application -n argocd demo-platform
kubectl get ns <request-name>
kubectl get deploy,svc,ingress -n <request-name>
kubectl get vd,vm -n <request-name>

```

Production evolution

Дальше этот сценарий можно развивать:

- backend, который создаёт branch и PR в Gitea/GitHub;
- CI validation request'ов;
- генератор manifests;
- TTL cleanup;
- интеграция с Vault/External Secrets;
- status page для разработчика.

Self-service portal

docs/self-service-portal.ru.md

Этот вариант показывает developer-facing портал, размещённый в DKP-кластере.

Адрес

```
https://selfservice-practicum.d8case.ru
```

Если DNS не настроен, добавьте на рабочей машине:

```
192.168.2.31 selfservice-practicum.d8case.ru
```

Архитектура

Диаграмма Mermaid преобразована в читаемую цепочку:

1	Разработчик
2	Ingress selfservice-practicum.d8case.ru
3	DexAuthenticator
4	Dex / OIDC
5	Self-service portal
6	Gitea API
7	GitOps repo
8	Argo CD
9	Namespace, app, ingress, DVP VM
10	Kubernetes status API

Разработчик → Ingress selfservice-practicum.d8case.ru → DexAuthenticator → Dex / OIDC → Self-service portal → Gitea API → GitOps repo → Argo CD → Namespace, app, ingress, DVP VM → Kubernetes status API

Пользователи и группы

На стенде practicum созданы три пользователя:

Пользователь	E-mail	Группа	Доступные профили
alice-koroleva-practicum	alice.koroleva.practicum@demo.local	practicum-payments-devs	app-only, app-with-vm
boris-smirnov-practicum	boris.smirnov.practicum@demo.local	practicum-analytics-devs	app-only, app-with-postgres-vm
marina-volkova-practicum	marina.volkova.practicum@demo.local	practicum-qa-devs	все профили

Рабочие пароли не хранятся в Git. Для текущего стенда они сохранены локально в ignored-файле `local/practicum-demo-users.env`.

Кнопка Выйти в шапке открывает /logout. Этот endpoint создаёт DexAuthenticator: он завершает текущую Dex-сессию и позволяет войти под другим демонстрационным пользователем.

Как работает заявка

- Пользователь входит через Dex.
- Portal получает X-Auth-Request-User, X-Auth-Request-Email, X-Auth-Request-Groups.
- Portal сопоставляет проверенный e-mail с allowlist пользователей и оставляет только разрешённые для него группы. Технический идентификатор из X-Auth-Request-User не используется как owner.
- Пользователь выбирает только разрешённый профиль, purpose и TTL.
- Имя стенда генерируется автоматически:

```
dev-<user>-<purpose>-<short-id>
```

- Backend создаёт в Gitea только EnvironmentRequest:

```
gitops/self-service/practicum/requests/<name>.yaml
```

- Request controller валидирует заявку и создаёт generated manifests.
- Argo CD применяет generated manifests в namespace practicum-tks.
- Portal читает status-файл, сформированный controller, включая причину Rejected или Error.

Что показывает UI

Для каждого профиля портал показывает:

- человеческое описание назначения профиля;
- список создаваемых ресурсов;
- квоты namespace;
- характеристики приложения;
- характеристики VM, если профиль создаёт DVP VM;
- роль AWX post-configuration для VM-профилей.

Назначение стенда тоже описывается явно:

Purpose	Что означает
feature	Проверка новой функциональности в изолированном namespace.
bugfix	Воспроизведение дефекта и проверка исправления.
loadtest	Короткий нагрузочный или ресурсный тест в рамках квот.
demo	Стенд для показа заказчику, команде или архитектурной аудиторией.

После отправки заявки portal показывает:

- пользовательский статус В работе, В очереди, Готово, Ошибка или Удалено по TTL;
- текущий этап обработки и время с момента отправки;
- обновление статуса каждые 5 секунд, пока операция не завершена;
- имя заявки;
- имя namespace и его phase;
- owner;

- профиль и purpose;
- TTL;
- квоты;
- параметры Deployment/demo-app;
- Service и Ingress;
- параметры VirtualDisk и VirtualMachine, если VM создаётся;
- пути GitOps artifacts в Gitea;
- URL приложения.

Кнопка отправки блокируется до завершения текущей операции, чтобы двойной клик не создал две одинаковые заявки. При временной ошибке чтения статуса portal показывает Ошибка связи и повторяет запрос через 5 секунд.

Если AWX job завершается неуспешно, controller выполняет не более трёх попыток. После исчерпания попыток заявка переходит из Provisioning в Error; portal показывает причину, номер последнего AWX job и прекращает polling.

Для профиля app-with-postgres-vm пользователь выбирает PostgreSQL 16, 17 или 18. Portal сохраняет выбор в заявке:

```
spec:
  profile: app-with-postgres-vm
  postgresql:
    version: "17"
```

Controller принимает только значения из allowlist, передаёт AWX пакет postgresql17, а playbook перед установкой проверяет его наличие через `apk search -x`. Для совместимости заявки, созданные до появления этого поля, трактуются как PostgreSQL 18.

Важное ограничение демо

В текущей реализации portal пишет сразу в main репозитория Gitea. Это удобно для живого демо.

Production-like вариант должен создавать branch/PR, запускать policy validation, после чего выполнять merge.

Проверка

```
kubectl get dexauthenticator -n practicum-tks selfservice-practicum
kubectl get certificate -n practicum-tks selfservice-practicum
kubectl get deploy,svc,ingress -n practicum-tks
kubectl get users.deckhouse.io \
  alice-koroleva-practicum boris-smirnov-practicum marina-volkova-practicum
kubectl get groups.deckhouse.io \
  practicum-payments-devs practicum-analytics-devs practicum-qa-devs
```

После создания стенда через UI:

```
kubectl get deploy,svc,ingress,vd,vm -n practicum-tks \
  -l demo.practicum/environment=<environment-id>
kubectl get application -n practicum-tks practicum-demo
```

Если wildcard DNS для *.d8case.ru отсутствует, добавьте hostname приложения на рабочей машине:

```
192.168.2.31 <environment-id>.d8case.ru
```

После этого приложение открывается по URL, который показывает portal.

IP виртуальной машины вида 10.66.x.x относится к внутренней DVP-сети. Для SSH не нужен маршрут с ноутбука: DVP CLI проксирует соединение через Kubernetes API. Portal выводит эти же параметры в

результате заявки:

- пользователь: `ansible`;
- аутентификация: SSH key;
- локальный ключ: `local/practicum-ssh/id_ed25519`;
- готовая команда с фактическим именем VM.

```
d8 v ssh ansible@<environment-id>-vm \  
--namespace practicum-tks \  
--identity-file local/practicum-ssh/id_ed25519 \  
--local-ssh
```

Проверка без интерактивной сессии:

```
d8 v ssh ansible@<environment-id>-vm \  
--namespace practicum-tks \  
--identity-file local/practicum-ssh/id_ed25519 \  
--local-ssh \  
--command 'hostname; whoami; uname -a'
```

Публиковать SSH через NodePort или LoadBalancer для этого демо не нужно. Пароль и приватный ключ не записываются в `EnvironmentRequest` или `status`: пользователь получает команду, а файл ключа подготавливается на ноутбуке демонстратора вне Git.

Ограничение ёмкости

Максимальное число одновременно активных стендов и VM задаётся в `Deployment practicum-request-controller`:

```
- name: MAX_ACTIVE_ENVIRONMENTS  
  value: "7"  
- name: MAX_ACTIVE_VMS  
  value: "7"
```

Controller читает эти значения из окружения. Если лимит достигнут, заявка остаётся в Git со статусом `Queued` и причиной `capacity-limit`; после освобождения ресурса следующий `reconcile` продолжает её автоматически.

Управление жизненным циклом

Пользовательский портал содержит разделы `Новый стенд`, `Мои стенды` и `История`. Пользователь может:

- удалить собственную VM вместе с `VirtualDisk`, сохранив приложение;
- удалить собственный стенд целиком;
- наблюдать состояния `VMDeleting`, `Deleting` и `Cleaned`.

Кнопки не удаляют Kubernetes-объекты напрямую. Backend создаёт `EnvironmentAction` в `gitops/self-service/practicum/actions/`. Controller проверяет владельца и атомарно обновляет `request`, `generated manifests` и `root customization` через Gitea `repoChangeFiles` API. Argo CD `prune` выполняет фактическое удаление.

Административный портал:

```
https://vm-admin-practicum.d8case.ru
```

Он доступен только Victor Melnikov и группе `practicum-vm-operators`. Victor видит `tenant environments` всех пользователей, может запускать, останавливать, перезапускать и удалять VM, а также удалять стенды. Для каждого действия обязательна причина. Golden builder VM и платформенные компоненты отфильтрованы на backend и не попадают в административный UI.

Для локального DNS:

```
192.168.2.31 vm-admin-practicum.d8case.ru
```

Demo talk track

docs/demo-talk-track.ru.md

Шпаргалка для живого показа стенда.

0. Вступление

Цель стенда: показать, как GitOps и AWX работают вместе.

Сразу зафиксировать границу:

- Argo CD не управляет ОС по SSH.
- Argo CD управляет Kubernetes/DVP-ресурсами из Git.
- AWX/Ansible управляет настройкой внутри ОС после создания workload'а или VM.

В локальном сценарии используются Linux pod'ы. В DKP/DVP сценарии есть настоящая минимальная VM postgres-vm.

1. Git: единый источник истины

Открыть:

```
http://gitea-awx.d8.kir.lab
```

Показать репозиторий codex/demo.

Файлы для показа:

- gitops/environments/prod/dvp-postgres-vm.yaml;
- gitops/environments/prod/demo-app.yaml;
- gitops/environments/prod/tenants/customer-a/;
- gitops/awx/playbooks/;
- scenarios/.

Что сказать:

Здесь лежит источник истины для демо: Kubernetes manifests, DVP manifests, Ansible playbooks и сценарии изменений. Все изменения можно обсуждать, ревьюить и откатывать через Git.

2. Argo CD: доставка ресурсов

Открыть:

```
http://argocd-awx.d8.kir.lab
```

Показать:

- Application ansible-os-pods;
- Application demo-platform;
- состояние Synced/Healthy;
- дерево ресурсов.

Команды:

```
kubectl get application -n argocd
kubectl get deploy,svc,ingress -n demo-prod
```

Что сказать:

Argo CD сравнивает кластер с Git и приводит его к описанному состоянию. Он не выполняет shell-команды внутри ОС, а синхронизирует Kubernetes API и DVP CRD.

3. DVP VM: инфраструктура из Git

Команда:

```
kubectl get vi,vd,vm -n demo-prod -o wide
```

Ожидаемо:

demo-alpine-cloud	Ready	
postgres-vm-root	Ready	256Mi
postgres-vm	Running	1 core 5% 512Mi

Что сказать:

Это настоящая DVP VM, но с минимальными ресурсами для стенда. Image, disk и VM описаны в Git и синхронизированы Argo CD.

4. AWX: настройка ОС

Открыть:

```
http://awx-demo.d8.kir.lab
```

Запустить:

```
AWX_URL=http://awx-demo.d8.kir.lab ./scripts/run-demo-job.sh
```

Показать:

- `stdout job;`
- `Gathering Facts;`
- `marker /etc/ansible-managed-by-awx;`
- `recap failed=0.`

Что сказать:

Этот файл появился не из Kubernetes manifest. Его создал AWX через Ansible внутри ОС. Так мы разделяем platform state и OS configuration.

5. Drift correction

Команды:

```
kubectl get deploy demo-app -n demo-prod
kubectl scale deployment demo-app --replicas=10 -n demo-prod
argocd app get demo-platform
kubectl get deploy demo-app -n demo-prod
```

Что сказать:

Ручное изменение в кластере является drift. При включенном self-heal Argo CD возвращает состояние к Git.

6. Rollback

Показать сценарий:

```
scenarios/06-broken-release-and-rollback.md
```

Что сказать:

Rollback не требует ручного исправления в кластере. Мы откатываем Git commit, и Argo CD приводит кластер к предыдущему рабочему состоянию.

7. Tenant onboarding

Команды:

```
kubectl get ns customer-a
kubectl get resourcequota,limitrange,rolebinding -n customer-a
kubectl get deploy,svc,ingress -n customer-a
```

Что сказать:

Новый tenant создаётся добавлением каталога в Git. Platform team контролирует шаблон, а onboarding становится повторяемым.

8. Финал

Если аудитория спрашивает про self-service для разработчиков, покажите:

```
self-service-ui/index.html
gitops/self-service/catalog/
gitops/self-service/requests/<request-name>.yaml
gitops/self-service/generated/<request-name>/
scenarios/09-self-service-environment-request.md
```

Что сказать:

Разработчик не получает прямой доступ к кластеру. Он выбирает approved profile в web UI или YAML request. UI генерирует GitOps request, дальше работают review, Argo CD и AWX. Это self-service без обхода governance.

Если аудитория спрашивает про golden image, покажите:

```
gitops/environments/prod/golden-images/source-image.yaml
scenarios/08-golden-image-management.md
```

Что сказать:

В production часто нужно управлять золотыми образами. В этом стенде администратор задаёт URL исходного image в Git. Argo CD создаёт DVP VirtualImage, DVP импортирует образ, AWX готовит builder VM, а новая версия golden image публикуется как отдельный immutable artifact.

Короткий порядок показа:

- Gitea: gitops/environments/prod/golden-images/source-image.yaml — URL исходного image.
- Argo CD: demo-platform — VirtualImage, VirtualDisk, VirtualMachine.
- Terminal: kubectl get vi,vd,vm -n demo-prod -o wide.
- AWX: prepare-golden-image.yaml и validate-golden-image.yaml.
- Gitea: publish-golden-image.example.yaml — публикация версии после validation.
- Объяснить rollback: переключение обратно на предыдущий image reference через Git.

Финальная формулировка:

Argo CD отвечает за декларативное состояние платформы. AWX отвечает за операционную настройку ОС. DVP VM, Kubernetes workload'ы и tenant'ы управляются из Git, а post-configuration и validation выполняются через AWX.

Prerequisites

docs/prerequisites.ru.md

Этот документ описывает требования к новому стенду DKP/DVP, на который переносится демо Argo CD + AWX + GitOps + DVP + self-service portal.

Целевая модель

На новом стенде должны быть доступны:

- Kubernetes API, управляемый через `kubectl`;
- DKP с включенными базовыми модулями для Ingress, Dex/OIDC и сертификатов;
- DVP с CRD `virtualization.deckhouse.io`;
- StorageClass для Gitea, AWX PostgreSQL, AWX projects PVC и DVP VirtualDisk;
- DNS или `/etc/hosts` записи для Gitea, Argo CD, AWX и self-service portal;
- egress до внешних registry и URL образов, если образы не зеркалируются во внутренний контур;
- права администратора кластера на время установки стенда.

Рабочая станция оператора

На машине, с которой выполняется перенос, должны быть установлены:

Инструмент	Для чего нужен	Проверка
<code>kubectl</code>	Управление кластером и проверка ресурсов	<code>kubectl version --client</code>
<code>git</code>	Работа с GitHub/Gitea и перенос репозитория	<code>git --version</code>
<code>curl</code>	Вызовы API Gitea/AWX	<code>curl --version</code>
<code>jq</code>	Разбор JSON в bootstrap-скриптах	<code>jq --version</code>
<code>base64</code>	Декодирование Kubernetes Secret	<code>base64 --help</code>
<code>sed</code> , <code>rg</code> или <code>grep</code>	Массовая проверка hardcoded значений	<code>rg --version</code> или <code>grep --version</code>

Доступ к кластеру:

```
kubectl config get-contexts
kubectl config current-context
kubectl get nodes -o wide
```

Kubernetes и DKP

Минимальные требования:

Требование	Зачем нужно	Проверка		
Рабочий kube-context	Все скрипты используют Kubernetes API	<code>kubectl get nodes</code>		
Права cluster-admin на установку	Создаются CRD, ClusterRole, ClusterRoleBinding, namespaces	<code>kubectl auth can-i '*' '*' --all-namespaces</code>		
Ingress controller с class nginx или адаптированный class	Публикация Gitea, Argo CD, AWX, portal, demo-app	<code>kubectl get ingressclass</code>		

Требование	Зачем нужно	Проверка		
DKP Dex/OIDC	Авторизация self-service portal через DexAuthenticator	`kubectl api-resources`	<code>grep -i dexauthenticator`</code>	
DKP User/Group CRD, если используются демо-пользователи	Создание alice-koroleva, boris-smirnov, marina-volkova	`kubectl api-resources`	<code>grep -E 'users`</code>	groups"
cert-manager или DKP certificate support	TLS для selfservice-* и корректный Dex callback	`kubectl api-resources`	<code>grep certificates`</code>	
Рабочий DNS или возможность обновить /etc/hosts	Доступ к UI и Ingress hostnames	<code>kubectl get ingress -A`</code>		

Если на целевом стенде IngressClass называется не nginx, нужно заменить его в:

```
manifests/dkp/ingresses.yaml
gitops/environments/prod/demo-app.yaml
gitops/environments/prod/tenants/customer-a/app.yaml
gitops/self-service/portal/ingress.yaml
gitops/self-service/portal/dex-authenticator.yaml
gitops/self-service/portal/app/server.py
gitops/self-service/generated/*/app.yaml
```

DVP

На стенде должен быть включен DVP и доступны CRD:

```
kubectl api-resources | grep -E 'virtualmachines|virtualdisks|virtualimages|clustervirtualimages'
```

Ожидаемые API-объекты:

- VirtualImage;
- ClusterVirtualImage;
- VirtualDisk;
- VirtualMachine;
- VirtualMachineClass.

Проверка класса VM:

```
kubectl get virtualmachineclass
```

В текущих манифестах используется:

```
virtualMachineClassName: generic
```

Если на целевом стенде нет класса generic, заменить его в:

```
gitops/environments/prod/dvp-postgres-vm.yaml
gitops/environments/prod/golden-images/builder-vm.yaml
gitops/self-service/portal/app/server.py
gitops/self-service/generated/*/vm.yaml
```

Storage

В текущем стенде используется StorageClass:

```
k8nfs
```

Он указан явно в:

```
manifests/awx/awx.yaml
manifests/awx/projects-pvc.yaml
gitops/environments/prod/dvp-postgres-vm.yaml
gitops/environments/prod/golden-images/builder-vm.yaml
gitops/self-service/portal/deployment.yaml
gitops/self-service/portal/app/server.py
gitops/self-service/generated/*/vm.yaml
```

На новом стенде нужно либо создать StorageClass k8nfs, либо заменить значение на фактический StorageClass.

Проверка:

```
kubectl get storageclass
kubectl get storageclass k8nfs
```

Рекомендации по объему:

Потребитель	Запрос в манифестах	Комментарий
Gitea PVC	2Gi	Репозиторий и sqlite data
AWX PostgreSQL	4Gi	В manifests/awx/awx.yaml
AWX projects PVC	2Gi	Сейчас отдельный пример PVC
postgres-vm-root	256Mi	Минимальный DVP disk
golden-builder-root	256Mi	Builder disk, VM остановлена вручную
Self-service VM disk	256Mi на VM	По одной VM на профиль app-with-vm

Практический минимум для чистого демо: 20Gi свободного persistent storage. Комфортный запас: 50Gi и больше, особенно если планируются дополнительные DVP images и несколько self-service VM.

Сетевой доступ и registry

Кластеру нужен egress до:

Назначение	Для чего
github.com	AWX operator kustomize install и внешний GitHub remote
quay.io	AWX EE и auxiliary images
gitea/gitea registry	Gitea image
dl-cdn.alpinelinux.org	Импорт Alpine cloud image для DVP
GitHub kirka1206/ArgoAWXk8sDVPdemo	Клонирование исходного проекта, если используется GitHub

Если стенд закрытый, заранее зеркалировать images и cloud image во внутренние registry/HTTP storage, затем заменить URL в:

```
gitops/environments/prod/dvp-postgres-vm.yaml
gitops/environments/prod/golden-images/source-image.yaml
gitops/environments/prod/golden-images/cluster-source-image.yaml
scripts/bootstrap.sh
manifests/gitea/gitea.yaml
manifests/awx/awx.yaml
```

DNS и имена

Текущий стенд использует домен:

```
d8.kir.lab
```

Текущие hostnames:

Host	Назначение
gitea-awx.d8.kir.lab	Gitea UI/API
argocd-awx.d8.kir.lab	Argo CD UI
awx-demo.d8.kir.lab	AWX UI/API
selfservice-awx.d8.kir.lab	Self-service portal
*.d8.kir.lab	Self-service app ingress hosts

На новом стенде выбрать новый домен, например:

```
demo.example.lab
```

И подготовить записи:

```
<INGRESS_IP> gitea-awx.demo.example.lab
<INGRESS_IP> argocd-awx.demo.example.lab
<INGRESS_IP> awx-demo.demo.example.lab
<INGRESS_IP> selfservice-awx.demo.example.lab
<INGRESS_IP> dev-alice-koroleva-feature-demo.demo.example.lab
```

Если wildcard DNS недоступен, для каждого self-service стенда нужно добавлять отдельную запись.

Git и репозитории

В текущем демо есть два remote:

Remote	Назначение
origin	GitHub kirka1206/ArgoAWXk8sDVPdemo
dkp-gitea	Внутренний Gitea стенда

Argo CD читает именно внутренний Gitea:

```
http://gitea-http.gitea.svc.cluster.local:3000/codex/demo.git
```

Для нового стенда нужно сохранить owner/repo codex/демо или поменять repoURL в:

```
manifests/argocd/application-demo.yaml
manifests/argocd/application-demo-platform.yaml
scripts/bootstrap.sh
gitops/self-service/portal/deployment.yaml
```

Секреты и пользователи

В Git не должны попадать реальные пароли и токены.

Перед запуском self-service portal нужно создать Secret для доступа backend к Gitea:

```
kubectl create namespace self-service-portal --dry-run=client -o yaml | kubectl apply -f -
kubectl -n self-service-portal create secret generic self-service-portal-gitea \
  --from-literal=username=codex \
  --from-literal=password='<gitea-password>' \
  --dry-run=client -o yaml | kubectl apply -f -
```

Demo users для Dex задаются через CRD User и Group. В Git есть только пример:

```
gitops/self-service/portal/dex-demo-users.example.yaml
```

Перед применением заменить `replace-with-bcrypt-hash` на реальные `bcrypt`-хэши или создать пользователей штатным способом целевого DKP.

AWX

Bootstrap создаёт pod-only AWX объекты:

- Project Gitea demo repo;
- Inventory Demo OS pods;
- Credential demo-pod-ssh;
- Job Template Configure OS pods.

Для DVP VM и golden image сценариев после разворачивания нужно вручную или отдельным скриптом создать:

- Inventory DVP VMs;
- Host postgres-vm с актуальным `ansible_host`;
- Credential dvp-vm-ssh;
- Job Template Bootstrap DVP VM;
- Inventory Golden Image Builders;
- Host golden-builder;
- Job Template Prepare Golden Image;
- Job Template Validate Golden Image.

AWX подключается к Git как Project и берет playbooks из:

```
gitops/awx/playbooks/
```

Проверка готовности стенда

Перед переносом выполнить:

```
kubectl get nodes -o wide
kubectl get storageclass
kubectl get ingressclass
kubectl api-resources | grep -E 'applications|virtualmachines|virtualdisks|virtualimages|dexauth
    enticator|certificates'
kubectl auth can-i '*' '*' --all-namespaces
```

Ожидаемый результат:

- Kubernetes API доступен;
- есть подходящий StorageClass;
- есть IngressClass;
- DVP CRD доступны;
- DKP Dex/cert-manager CRD доступны, если включается self-service portal;
- оператор имеет права на создание namespaces, CRD-backed resources и RBAC.

Migration plan

docs/migration-plan.ru.md

Документ описывает перенос текущего демо Argo CD + AWX + Gitea + DKP/DVP + self-service portal на новый стенд.

Перед началом прочитайте:

```
docs/prerequisites.ru.md
docs/STATUS.md
docs/NEXT_STEPS.md
```

0. Зафиксировать исходное состояние

На текущем стенде:

```
kubectl config current-context
kubectl get application -n argocd -o wide
kubectl get ns
kubectl get vi,cvi,vd,vm -A -o wide
git status --short --branch
git log --oneline -5
```

Ожидаемо для актуального состояния проекта:

```
ansible-os-pods    Synced    Healthy
demo-platform     Synced    Healthy
```

Текущая живая ревизия после последних исправлений:

```
13e30f798f30d77877bc6206659d7abe3fe397b1
```

Рекомендуется создать тег или отдельную ветку перед переносом:

```
git tag -a migration-ready-$(date +%Y%m%d) -m "Migration-ready demo state"
git push origin --tags
git push dkp-gitea --tags
```

1. Выбрать параметры нового стенда

До изменения файлов заполнить таблицу:

Параметр	Текущий стенд	Новый стенд
kube-context	codex-api.d8.kir.lab	<target-context>
Базовый домен	d8.kir.lab	<target-domain>
Ingress IP	10.77.77.208	<target-ingress-ip>
Gitea host	gitea-awx.d8.kir.lab	gitea-awx.<target-domain>
Argo CD host	argocd-awx.d8.kir.lab	argocd-awx.<target-domain>
AWX host	awx-demo.d8.kir.lab	awx-demo.<target-domain>
Portal host	selfservice-awx.d8.kir.lab	selfservice-awx.<target-domain>
StorageClass	k8nfs	<target-storage-class>
DVP VM class	generic	<target-vm-class>
Gitea owner/repo	codex/demo	codex/demo или новое значение

Решение: если хотите минимизировать правки, на новом стенде сохранить codex/demo, k8nfs, generic и hostnames по тому же шаблону, поменяв только домен.

2. Подготовить целевой DKP/DVP кластер

Проверить доступ:

```
kubectl config use-context <target-context>
kubectl get nodes -o wide
kubectl get storageclass
kubectl get ingressclass
```

Проверить DVP:

```
kubectl api-resources | grep -E 'virtualmachines|virtualdisks|virtualimages|clustervirtualimages'
kubectl get virtualmachineclass
```

Проверить DKP/Dex/certificates:

```
kubectl api-resources | grep -E 'dexauthenticator|certificates|users|groups'
```

Если чего-то нет, сначала включить нужные DKP modules или подготовить альтернативный вариант:

- без self-service portal DexAuthenticator;
- без DVP сценариев;
- с другим IngressClass;
- с другим StorageClass.

3. Подготовить DNS или /etc/hosts

Для временного переноса можно использовать /etc/hosts:

```
<target-ingress-ip> gitea-awx.<target-domain>
<target-ingress-ip> argocd-awx.<target-domain>
<target-ingress-ip> awx-demo.<target-domain>
<target-ingress-ip> selfservice-awx.<target-domain>
```

Для self-service apps желательно wildcard DNS:

```
*.<target-domain> -> <target-ingress-ip>
```

Если wildcard невозможен, добавлять записи для создаваемых стендов:

```
<target-ingress-ip> dev-alice-koroleva-demo-c3aa.<target-domain>
<target-ingress-ip> dev-alice-koroleva-feature-f72b.<target-domain>
<target-ingress-ip> dev-alice-koroleva-feature-8c3e.<target-domain>
```

4. Склонировать проект и создать migration branch

На рабочей машине:

```
git clone git@github.com:kirka1206/ArgoAWXk8sDVPdemo.git
cd ArgoAWXk8sDVPdemo
git checkout main
git pull origin main
git checkout -b migrate/<target-stand-name>
```

Если переносите именно текущее состояние из Gitea стенда, добавьте remote и подтяните его:

```
git remote add source-dkp-gitea http://gitea-awx.d8.kir.lab/codex/demo.git || true
git fetch source-dkp-gitea main
git merge --no-edit source-dkp-gitea/main
```

5. Заменить параметры стенда

Найти текущие hardcoded значения:

```
rg -n 'd8\\.kir\\.lab|codex-api\\.d8\\.kir\\.lab|10\\.77\\.77\\.208|k8nfs|virtualMachineClassNam
e: generic' .
```

Обновить минимум:

```
scripts/deploy-dkp.sh
manifests/dkp/ingresses.yaml
gitops/self-service/portal/ingress.yaml
gitops/self-service/portal/certificate.yaml
gitops/self-service/portal/dex-authenticator.yaml
gitops/self-service/portal/deployment.yaml
gitops/environments/prod/dvp-postgres-vm.yaml
gitops/environments/prod/golden-images/builder-vm.yaml
gitops/self-service/generated/*/app.yaml
gitops/self-service/generated/*/vm.yaml
```

Если меняется owner/repo Gitea, обновить:

```
manifests/argocd/application-demo.yaml
manifests/argocd/application-demo-platform.yaml
gitops/self-service/portal/deployment.yaml
```

Если меняется StorageClass, заменить:

```
rg -n 'k8nfs' .
```

Если меняется домен self-service apps, заменить:

```
rg -n 'd8\\.kir\\.lab' gitops/self-service manifests/dkp docs README.md README.ru.md
```

6. Решить, переносить ли текущие generated self-service стенды

В текущем проекте есть live-generated заявки:

```
gitops/self-service/requests/dev-alice-koroleva-demo-c3aa.yaml
gitops/self-service/requests/dev-alice-koroleva-feature-f72b.yaml
gitops/self-service/requests/dev-alice-koroleva-feature-8c3e.yaml
```

Они демонстрационно полезны, потому что показывают результат работы portal backend.

Есть два варианта.

Вариант А. Перенести как есть

Оставить requests/ и generated/. На новом стенде Argo CD создаст эти namespace и, для профилей с VM, DVP VM.

Плюс: быстро видно готовый результат.

Минус: это уже созданные демо-заявки Alice, их TTL логически истек или скоро истечет. В текущей реализации TTL cleanup automation еще не реализован.

Вариант В. Очистить перед переносом

Удалить generated-заявки из Git:

```
git rm -r gitops/self-service/requests/dev-alice-koroleva-*.yaml
git rm -r gitops/self-service/generated/dev-alice-koroleva-*
```

Оставить валидный пустой top-level kustomization:

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources: []
```

Плюс: новый стенд стартует чистым.

Минус: результат self-service нужно будет создать заново через portal.

7. Проверить рендер manifests до установки

Локально:

```
kubectl kustomize gitops/environments/prod >/tmp/demo-platform.yaml
kubectl kustomize gitops/demo-manifests >/tmp/ansible-os-pods.yaml
```

Если установлен kustomize:

```
kustomize build gitops/environments/prod >/tmp/demo-platform.yaml
```

Проверить, что нет старого домена:

```
rg -n 'd8\\.kir\\.lab|10\\.77\\.77\\.208|codex-api\\.d8\\.kir\\.lab' .
```

Допустимо оставить старые значения только в исторической документации, если она явно помечена как пример исходного стенда.

8. Подготовить и отправить Git

Закоммитить изменения migration branch:

```
git status --short
git add .
git commit -m "Adapt demo for <target-stand-name>"
git push origin migrate/<target-stand-name>
```

Дальше либо merge в main, либо отдельная ветка для проверки.

Для live Argo CD в новом стенде важно, чтобы проект попал во внутренний Gitea этого стенда. Bootstrap делает это автоматически через local-gitea.

9. Запустить базовый bootstrap на новом стенде

Выполнить:

```
CONTEXT=<target-context> \
GITEA_HOST=gitea-awx.<target-domain> \
ARGOCD_HOST=argocd-awx.<target-domain> \
AWX_HOST=awx-demo.<target-domain> \
./scripts/deploy-dkp.sh
```


Скрипт:

- проверит kube-context;
- установит Argo CD;
- установит Gitea;
- создаст пользователя codex и репо demo;
- отправит текущий проект во внутренний Gitea;
- создаст Argo CD Application `ansible-os-pods`;
- установит AWX operator и AWX;
- создаст базовые AWX объекты для pod-only сценария;
- создаст Ingress для Gitea, Argo CD и AWX.

Проверка:

```
kubectl get pods -n argocd
kubectl get pods -n gitea
kubectl get pods -n awx
kubectl get application -n argocd ansible-os-pods
```

10. Применить расширенный DVP Application

После bootstrap:

```
kubectl apply -f manifests/argocd/application-demo-platform.yaml
kubectl annotate application -n argocd demo-platform argocd.argoproj.io/refresh=hard --overwrite
```

Ждать:

```
kubectl get application -n argocd demo-platform -w
```

Проверить:

```
kubectl get deploy,svc,ingress -n demo-prod
kubectl get vi,cvi,vd,vm -A -o wide
kubectl get ns customer-a
```

Ожидаемый минимум:

- `demo-platform` в Synced/Healthy;
- `demo-prod/demo-app` доступен;
- `VirtualImage/demo-alpine-cloud` в Ready;
- `VirtualDisk/postgres-vm-root` в Ready;
- `VirtualMachine/postgres-vm` в Running;
- `ClusterVirtualImage/alpine-base-3-23-v1` в Ready.

11. Настроить DNS и проверить UI

Проверить:

```
curl -I http://gitea-awx.<target-domain>
curl -I http://argocd-awx.<target-domain>
curl -I http://awx-demo.<target-domain>
```

Открыть в браузере:

```
http://gitea-awx.<target-domain>
http://argocd-awx.<target-domain>
http://awx-demo.<target-domain>
```

Argo CD admin password:

```
kubectl -n argocd get secret argocd-initial-admin-secret \
-o jsonpath='{.data.password}' | base64 -d; echo
```

AWX admin password:

```
kubectl -n awx get secret awx-demo-admin-password \
-o jsonpath='{.data.password}' | base64 -d; echo
```

12. Подготовить self-service portal

Создать Secret для Gitea API:

```
kubectl create namespace self-service-portal --dry-run=client -o yaml | kubectl apply -f -
kubectl -n self-service-portal create secret generic self-service-portal-gitea \
--from-literal=username=codex \
--from-literal=password='<gitea-password>' \
--dry-run=client -o yaml | kubectl apply -f -
```

Создать demo users/groups, если они нужны:

```
cp gitops/self-service/portal/dex-demo-users.example.yaml /tmp/dex-demo-users.yaml
# заменить replace-with-bcrypt-hash на реальные bcrypt-хэши
kubectl apply -f /tmp/dex-demo-users.yaml
```

Проверить portal:

```
kubectl get deploy,svc,ingress,dexauthenticator,certificate -n self-service-portal
curl -kI https://selfservice-awx.<target-domain>
```

Открыть:

```
https://selfservice-awx.<target-domain>
```

13. Настроить AWX для DVP VM

Bootstrap создает pod-only AWX job template. Для DVP VM добавить:

- Inventory DVP VMs.
- Credential dvp-vm-ssh.
- Host postgres-vm с актуальным IP:

```
kubectl get vm postgres-vm -n demo-prod -o wide
```

- Job Template Bootstrap DVP VM:

```
Inventory: DVP VMs
Project: Gitea demo repo
Playbook: gitops/awx/playbooks/bootstrap-vm.yml
Credential: dvp-vm-ssh
```

- Job Template Validate DVP VM:

```
Inventory: DVP VMs
Project: Gitea demo repo
Playbook: gitops/awx/playbooks/validate-vm.yml
Credential: dvp-vm-ssh
```

Проверка доступа из кластера:

```
VM_IP=$(kubectl get vm postgres-vm -n demo-prod -o jsonpath='{.status.ipAddress}')
kubectl run -n demo-prod netcheck --rm -i --restart=Never \
  --image=busybox:1.36 --timeout=20s -- \
  nc -vz -w 3 "$VM_IP" 22
```

14. Настроить golden image сценарий

Проверить source image:

```
kubectl get vi alpine-base-3-23-v1 -n demo-prod
kubectl get cvi alpine-base-3-23-v1
kubectl get vd,vm -n demo-prod | grep golden
```

Для запуска builder VM:

```
kubectl patch vm golden-builder-vm -n demo-prod --type merge -p
'{"spec":{"runPolicy":"AlwaysOn"}}'
```

После получения IP добавить host в AWX inventory Golden Image Builders:

```
kubectl get vm golden-builder-vm -n demo-prod -o wide
```

Создать в AWX:

```
Job Template: Prepare Golden Image
Inventory: Golden Image Builders
Project: Gitea demo repo
Playbook: gitops/awx/playbooks/prepare-golden-image.yml
Credential: dvp-vm-ssh

Job Template: Validate Golden Image
Inventory: Golden Image Builders
Project: Gitea demo repo
Playbook: gitops/awx/playbooks/validate-golden-image.yml
Credential: dvp-vm-ssh
```

Публикацию новой версии golden image выполнять через новый versioned manifest, не через перезапись существующего provisioned disk.

15. Проверить демонстрационные сценарии

Минимальный smoke test:

```
kubectl get application -n argocd
kubectl get pods -n demo-os
kubectl get deploy,svc,ingress -n demo-prod
kubectl get vi,cvi,vd,vm -A -o wide
AWX_URL=http://awx-demo.<target-domain> ./scripts/run-demo-job.sh
```

Self-service portal smoke test:

- Войти как demo user через Dex.
- Создать app-only стенд.
- Дождаться commit в Gitea.
- Проверить demo-platform в Argo CD.

- Проверить namespace:

```
kubectl get ns | grep dev-
kubectl get deploy,svc,ingress -n <generated-namespace>
```

VM profile smoke test:

```
kubectl get vd,vm -n <generated-namespace> -o wide
```

16. Критерии приемки переноса

Перенос считается успешным, если:

- Gitea UI открывается по новому host;
- Argo CD UI открывается по новому host;
- AWX UI открывается по новому host;
- ansible-os-pods в Synced/Healthy;
- demo-platform в Synced/Healthy;
- demo-os pod-only сценарий проходит через AWX;
- demo-prod/postgres-vm в Running;
- ClusterVirtualImage/alpine-base-3-23-v1 в Ready;
- self-service portal открывается через HTTPS и Dex;
- portal может создать request в Gitea;
- Argo CD применяет generated namespace;
- для VM-профиля DVP создает VirtualDisk и VirtualMachine.

17. Rollback и очистка

Если перенос не удался до применения demo-platform:

```
./scripts/destroy.sh
```

Если demo-platform уже применен:

```
kubectl delete application demo-platform -n argocd --cascade=foreground
kubectl delete ns demo-prod customer-a self-service-portal --ignore-not-found
```

Для generated self-service namespaces:

```
kubectl get ns | grep '^dev-'
kubectl delete ns <generated-namespace>
```

GitOps-правильный rollback:

```
git revert HEAD
git push origin main
git push dkp-gitea main
kubectl annotate application -n argocd demo-platform argocd.argoproj.io/refresh=hard --overwrite
```

18. Типовые проблемы при переносе

Argo CD не может собрать manifests

Проверить:

```
kubectl kustomize gitops/environments/prod
kubectl describe application -n argocd demo-platform
```

Частый случай для self-service:

```
resources: []
- some-env
```

Это невалидно. Должно быть:

```
resources:
- some-env
```

или пустой список:

```
resources: []
```

DVP запрещает менять VirtualDisk.dataSource

Если диск уже provisioned, DVP не разрешает менять источник. Используйте новый диск, restore/recreate flow или scoped ignoreDifferences, как сделано для demo-prod/postgres-vm-root.

Portal пишет в Git, но Argo CD ничего не создает

Проверить:

```
kubectl logs -n self-service-portal deploy/self-service-portal
kubectl get application demo-platform -n argocd
git fetch dkp-gitea main
git log --oneline dkp-gitea/main -5
```

Причины:

- portal пишет не в тот owner/repo/branch;
- Secret self-service-portal-gitea неверный;
- Argo CD смотрит на другой repoURL;
- generated kustomization.yaml не включает новый каталог.

Dex login возвращает 403

Проверить:

```
kubectl get dexauthenticator -n self-service-portal
kubectl get users.deckhouse.io
kubectl get groups.deckhouse.io
kubectl logs -n self-service-portal deploy/self-service-portal
```

Пользователь должен входить в одну из allowed groups:

```
payments-devs
analytics-devs
qa-devs
```

AWX не подключается к VM

Проверить IP и SSH:

```
kubectl get vm -A -o wide
kubectl run -n demo-prod netcheck --rm -i --restart=Never \
  --image=busybox:1.36 --timeout=20s -- \
  nc -vz -w 3 <vm-ip> 22
```

Проверить AWX credential и host variables.

Next steps

docs/NEXT_STEPS.md

Приоритет 1. Подготовить стенд к повторному показу

Для всех следующих прогонов использовать только `scenarios/d8case/README.md`. Корневые исторические сценарии не смешивать с командами `practicum`-стенда.

- Создать DNS-записи для `gitea-practicum.d8case.ru`, `argocd-practicum.d8case.ru`, `awx-practicum.d8case.ru` и `selfservice-practicum.d8case.ru` на `192.168.2.31`.
- Заменить `self-signed Certificate portal` на доверенный `issuer`.
- Alice проверена через реальный Dex UI: доступны только `app-only` и `app-with-vm`. Проверить Boris и Marina и соответствие их профилей группам.
- После доставки кнопки Выйти проверить завершение Dex-сессии, повторный `redirect` и вход под другим пользователем.
- Создать новое тестовое окружение через Web, дождаться Ready, после чего проверить корректный `owner/Environment ID`, переход `Submitted -> Provisioning -> Ready` и автоматический `TTL cleanup`.
- Проверить PostgreSQL-профиль для версий 16, 17, 18 и выполнить показанную `portal` команду `d8 v ssh`.
- Добавить `192.168.2.31 vm-admin-practicum.d8case.ru` в постоянный DNS; временная запись `/etc/hosts`, Dex-доступ Victor и `lifecycle API` уже проверены.
- Повторить `scenarios/13-manual-environment-lifecycle.md` как чистый демонстрационный прогон; техническая `live-приёмка start/stop/restart, delete VM и delete environment` уже выполнена.
- На следующем пользовательском прогоне проверить расширенную карточку Мои стенды, ручное обновление без `browser cache` и прогресс удаления с `polling` каждые 5 секунд.
- После исправления Victor `portal` убедиться, что алерт `IngressResponses5xx` закрылся после очистки окна метрики, и провести демонстрационный переход по всем пяти представлениям.
- Выполнить `scenarios/11-practicum-end-to-end.md`, не изменяя опубликованные `golden image v1/v2`.
- Заменить временный SuperAdmin пользователя `practicum-tks@demo.local` на минимально необходимые права.

Приоритет 2. Проверить роли и профили

- Alice: проверить `app-only` и `app-with-vm`.
- Boris: проверить `app-only` и `app-with-postgres-vm`.
- Marina: проверить все три профиля.
- Victor: проверить `start/stop/restart, console/VNC, disks и snapshots`.
- Подтвердить Forbidden для Victor на `Secrets, Deployments, RBAC` и изменение `VirtualImage/ClusterVirtualImage`.
- Подтвердить, что разработчики не могут создавать VM напрямую.

- Провести `scenarios/12-dvp-vm-drift-correction.md` на временной VM и проверить кратковременный `OutOfSync`, `self-heal` и `controlled restart`.

Приоритет 3. Усилить Git workflow

Repository webhook Gitea -> Argo CD настроен для немедленного refresh после push. Периодический polling остаётся резервным механизмом.

- Перевести automation с прямых commit в main на branch/PR.
- Атомарное обновление `action/generated/kustomization` через Gitea `героChangeFiles` реализовано; следующим шагом добавить `retry` с `exponential backoff` для конфликтов конкурентных commits.
- Добавить CI-проверку схемы `EnvironmentRequest`, `owner/group/profile/TTL` и `Kustomize render` до merge.
- Добавить защиту ветки `main` и обязательное review platform team.
- Исключить status-коммиты, если `semantic status` не изменился.

Приоритет 4. Production hardening

- Удалить `legacy demo-default codex123` из старых локальных bootstrap артефактов и заменить его обязательной генерацией/передачей `Secret`.
- Перенести `runtime credentials` в `External Secrets/Vault`.
- Добавить Prometheus-метрики controller: `queued`, `active`, `failed`, `cleanup`.
- Добавить `retry/backoff status` для временных ошибок Gitea/AWX.
- Добавить `NetworkPolicy` для `portal`, `controller`, `Gitea` и `AWX`.
- Добавить резервное копирование Gitea и AWX PostgreSQL/PVC.
- Связать `request commit`, `generated commit`, `Argo revision`, `AWX job` и `cleanup commit` в едином `audit trail`.
- Проверить `quota exhaustion` и очередь при настраиваемых лимитах `MAX_ACTIVE_ENVIRONMENTS=7`, `MAX_ACTIVE_VMS=7`.

Приоритет 5. Golden image lifecycle

- Описать выпуск `v3` без изменения `v1/v2`.
- Автоматизировать публикацию `image` только после успешной `validation`.
- Добавить `metadata`: `source URL/checksum`, `package list`, `build date`, `AWX job` и `Git revision`.
- Реализовать `controlled promotion candidate -> active`.
- Проверить `rollback activeGoldenImage` с `v2` на `v1` отдельным Git-коммитом.
- Решить, сохранять ли `builder disks` после демо; `versioned VirtualImage` удалять или изменять нельзя.

Порядок безопасного продолжения

- Проверить `context practicum-tks-api.d8case.ru`.
- Проверить `practicum-demo` в состоянии `Synced/Healthy`.
- Получить новые controller-коммиты из Gitea до локальных изменений.

- Не использовать force-push.
- Не удалять объекты без labels/prefix practicum.
- После каждого этапа обновлять docs/STATUS.md, запускать проверки и отправлять изменения в GitHub и practicum Gitea.

Полезные команды

```
kubectl get application -n practicum-tks practicum-demo
kubectl get vi,vd,vm -n practicum-tks -o wide
kubectl get deploy,svc,ingress -n practicum-tks
kubectl logs -n practicum-tks deploy/practicum-request-controller
kubectl get dexauthenticator,certificate -n practicum-tks
kubectl get deploy,svc,ingress,vd,vm -n practicum-tks \
  -l demo.practicum/environment
git fetch practicum-gitea main
git status
```